

Visualización estática de datos

Lección 4: Gráficos de distribución

Introducción

En la lección anterior, aprendiste a crear gráficos de líneas, barras y circulares para representar valores asociados a variables cualitativas o cuantitativas discretas y obtener su frecuencia; sin embargo, muchas de las magnitudes que utilizamos en nuestro día a día poseen valores continuos, es decir, que se miden en lugar de contarse. Por ejemplo, la temperatura, el peso, la velocidad de tu coche, el salario anual, el tiempo de entrega de tus pedidos y la presión arterial, son variables cuantitativas continuas.

En esta lección, revisarás las representaciones gráficas más usuales que permiten analizar este tipo de variables. Además, experimentarás la pertinencia de aplicar diversos tipos de gráficos para la exploración de características particulares de tus datos, con relación a su distribución. Esto te permitirá integrar en tus *scripts* de Python resúmenes visuales con grandes volúmenes de información, que te ayudarán a encontrar tendencias y relaciones cuya identificación se dificultaría en un formato tabular.

Lo anterior coloca a la fase de visualización en un peldaño importante dentro de la ciencia de datos y en la toma de decisiones basadas en los mismos.



Antes de comenzar, piensa en la información que manipulas en tu organización:

- ¿Qué tipo de valores posee?
- ¿Conoces sus rangos típicos?
- ¿Cuántas y cuáles variables están involucradas en tus interrogantes?
- ¿Puedes modelar sus comportamientos y/o relaciones?

¿Estás listo? ¡Comencemos!

Histogramas

Los histogramas se utilizan para mostrar datos cuantitativos continuos, con barras rectangulares de superficie proporcional a la frecuencia de los valores representados; ofrecen un panorama de la distribución de dichos valores, respecto a una variable. A diferencia del gráfico de barras, en el histograma las barras se dibujan juntas porque representan un espectro continuo de valores, los cuales se agrupan en intervalos que representan las clases (colocados usualmente en el eje X).

Emplea este tipo de diagrama cuando desees observar el grado de homogeneidad o variabilidad de las columnas cuantitativas continuas del *dataframe*.

Para ejemplificar su uso, utilizaremos el archivo **LifeExpectancyMale.csv** (localizado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente), que almacena el indicador de esperanza de vida al nacer de los hombres, por país, desde 1960. Los valores indican la cantidad de años que viviría un recién nacido si los patrones de mortalidad prevalecientes en el momento de su nacimiento fueran los mismos a lo largo de su vida.

Esta información se ha almacenado en el *dataframe* `lifexp_male` (265 filas x 66 columnas). En la siguiente imagen puedes observar que es un *dataframe* no etiquetado en las filas (el índice son las posiciones consecutivas que ocupa cada registro):

	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
0	Aruba	ABW	60.746000	61.114000	61.371000	...	73.047000	72.344000	70.794000	73.537000	73.702000
1	Africa Eastern and Southern	AFE	42.448496	42.681453	43.117691	...	61.289264	61.048093	60.351874	61.765707	62.395547
2	Afghanistan	AFG	32.136000	32.626000	33.098000	...	59.910000	58.455000	57.052000	63.941000	64.467000
3	Africa Western and Central	AFW	35.925673	36.181219	37.031000	...	56.145037	56.219168	56.296796	56.906135	57.752356
...	...	...	...	...	...	...	...	...	...	...	...
261	Yemen, Rep.	YEM	31.105000	31.720000	30.986000	...	63.337000	63.330000	63.090000	65.665000	67.235000
262	South Africa	ZAF	50.823000	51.188000	51.707000	...	62.696000	62.174000	59.193000	61.813000	62.609000
263	Zambia	ZMB	49.581000	49.998000	50.258000	...	60.349000	60.731000	59.790000	62.768000	63.936000
264	Zimbabwe	ZWE	51.077000	51.516000	51.965000	...	58.830000	59.091000	57.717000	59.880000	60.226000

265 rows × 66 columns

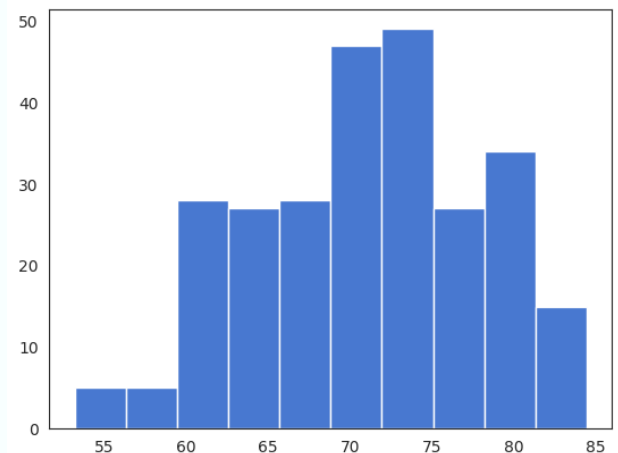
Para graficar un histograma con la expectativa de vida del año 2023, utilicemos varias de las formas equivalentes estudiadas:

Con matplotlib

```
plt.hist(lifexp_male['2023'])
```

O

```
plt.hist('2023',data=lifexp_male)
```



Se utilizó la paleta *muted*:

```
sns.set_palette('muted')
```

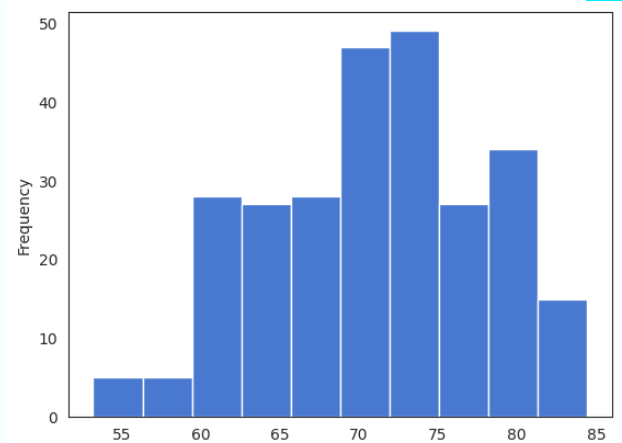
Para incluir las líneas de contorno utiliza el parámetro `edgecolor='black'`

Con pandas

```
lifexp_male['2023'].plot.hist()
```

O

```
lifexp_male['2023'].plot(kind='hist')
```



Para incluir las líneas de contorno utiliza el parámetro `edgecolor='black'`

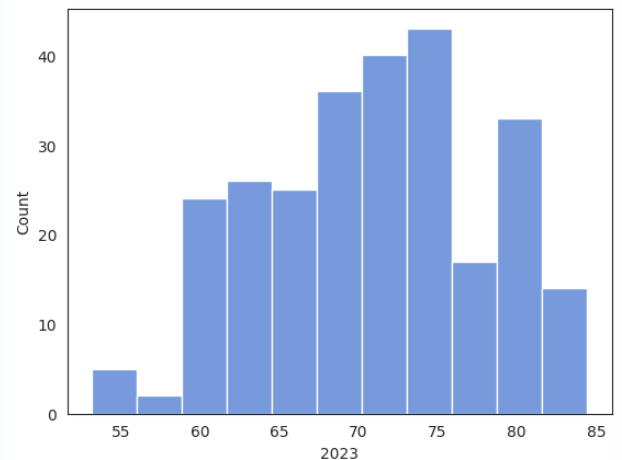
Para incluir las líneas de contorno utiliza el parámetro `edgecolor='black'`

Con seaborn

```
sns.histplot(x=lifexp_male['2023'])
```

O

```
sns.histplot(x='2023', data=lifexp_male)
```



Se utilizó la paleta *muted*: `sns.set_palette('muted')`

Como puedes observar, en matplotlib y pandas hay 10 barras. Eso quiere decir que los datos han sido agrupados en 10 clases (o *bins*), que es el valor por defecto cuando este número no es especificado en dichas plataformas. En cambio, seaborn utiliza un algoritmo de selección de *bins* (Freedman-Diaconis, que tiende a ajustar el número de *bins* a las características de los datos) por defecto, que es más sensible a la dispersión y el tamaño de los datos, resultando en un número variable de *bins* (para este conjunto, resultaron 11).

Las longitudes de las barras indican la frecuencia de aparición de los datos en cada clase. Así, por ejemplo, en la primera clase hay 5 países; en la tercera, cerca de 30 países.

La anchura de cada clase se calcula dividiendo el rango de valores (máximo - mínimo) entre el número de *bins*. Si haces el cálculo para determinar la anchura de los *bins* en los histogramas previos obtienes:

$$\frac{\text{lifexp_male['2023'].max() - lifexp_male['2023'].min()}}{10}$$
$$\frac{84.448 - 53.195}{10}$$
$$= 3.125$$

Lo anterior significa que los *bins* tienen los siguientes intervalos:

- 1^{er} *bin*: de 53.195 (*min*) a 56.32 (53.195+3.125) años
- 2^o *bin*: de 56.32 a 59.445 (56.32+3.125) años
- 3^{er} *bin*: de 59.445 a 62.57 (59.445+3.125) años

Así sucesivamente hasta llegar al valor máximo (*max*).

Matplotlib permite obtener directamente estos límites a través de los valores que devuelve la función `hist()`, la cual retorna, entre otros elementos, los bordes que definen cada segmento del histograma.

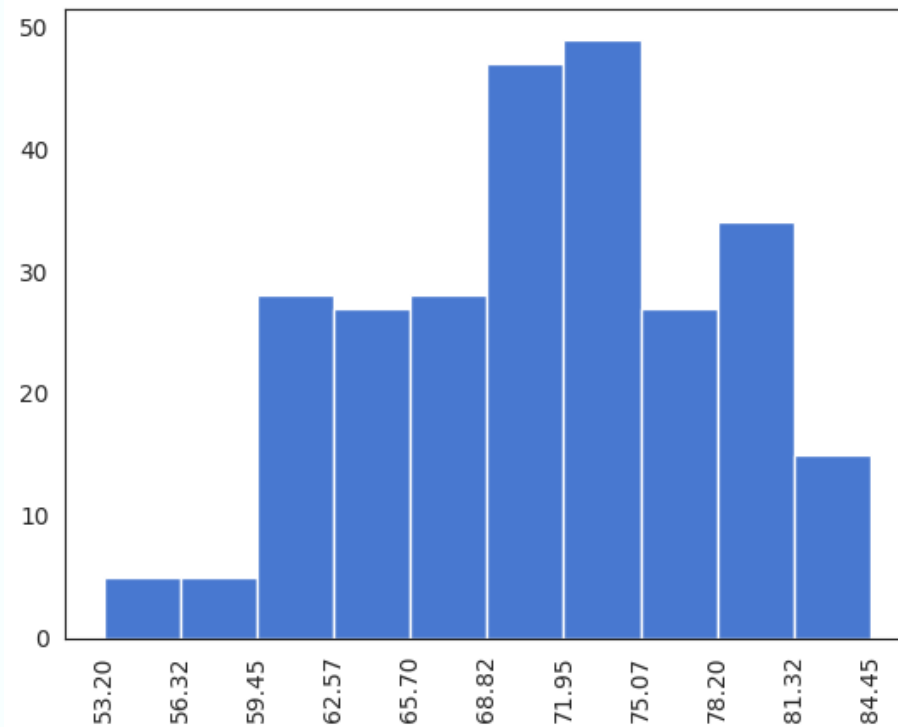
```
n, bins, edges = plt.hist(lifexp_male['2023'])
```

Donde:

- *n*: Es un arreglo con la cantidad de datos (frecuencia) que cayeron en cada *bin*; es decir, cuántos valores hay dentro de cada intervalo del histograma.
- *bins*: Es un arreglo con los límites de los intervalos. Si tienes *k* barras, este arreglo tendrá *k+1* valores, ya que representa los bordes izquierdo y derecho de cada *bin*.
- *patches*: Una lista de objetos tipo `matplotlib.patches.Rectangle`, que son las barras dibujadas en el histograma. Puedes utilizarlos para modificar colores, bordes, etc.

Veamos cómo incluir los límites en el gráfico:

```
n, bins, edges = plt.hist(lifexp_male['2023'])  
plt.xticks(bins)  
plt.xticks(rotation=90)
```

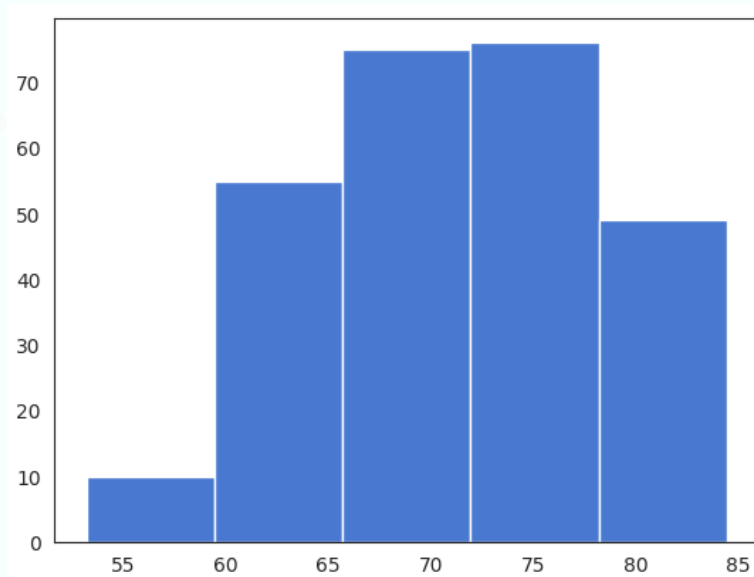


Como podrás observar, los valores coinciden con lo calculado anteriormente para los tres primeros *bins*.

También puedes especificar explícitamente la cantidad de *bins* que deseas mostrar, incluyendo el parámetro *bins* y **asignándole un número entero** (cantidad de bins igualmente espaciados) **o una secuencia** que define los bordes de los *bins* (en este caso, las clases pueden estar espaciadas de manera irregular). Probemos ambas opciones.

Por ejemplo, para crear únicamente 5 clases, basta con especificarlo con el parámetro *bins* de manera análoga en las tres plataformas:

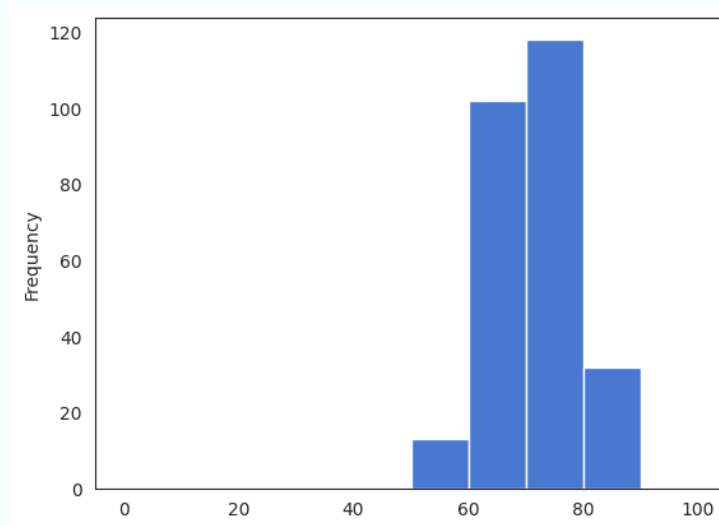
```
plt.hist('2023', data=lifexp_male, bins=5)
```



Si indicas una secuencia que define los bordes de las clases, por ejemplo, haciendo 10 clases, cada una con una anchura de 10 años, entonces:

```
lifexp_male['2023'].
plot(kind='hist',bins=[0,10,20,30,40,50,60,70,80,90,100])
o
lifexp_male['2023'].plot(kind='hist',bins=np.
arange(0,101,10))
```

Obteniendo lo siguiente:

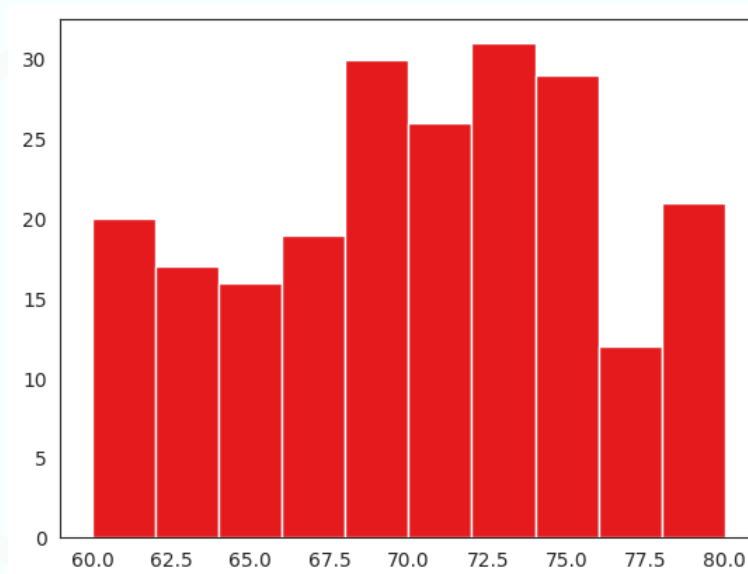


Las clases de este histograma van de [0,10), [10,20), [20,30), y así sucesivamente. El histograma obtenido muestra que la expectativa de vida en la mayoría de los países está entre los 70 y 80 años.

Otros parámetros de utilidad son:

- `range` (para matplotlib y pandas) y `binrange` (para seaborn): Establece el límite inferior y superior a considerar. Los valores fuera de este rango se ignoran.

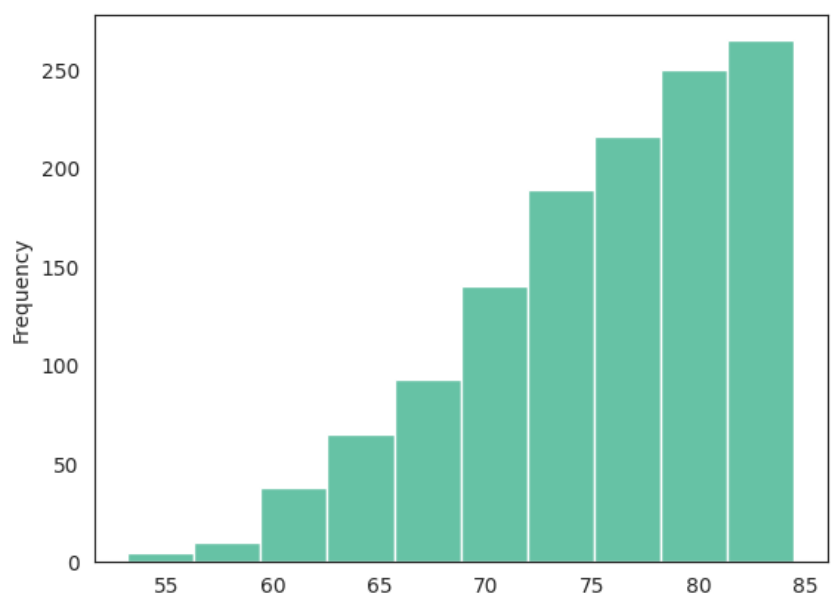
```
plt.hist('2023', data=lifexp_male, range=(60, 80))
```



Se empleó la paleta `Set1`: `sns.set_palette('Set1')`

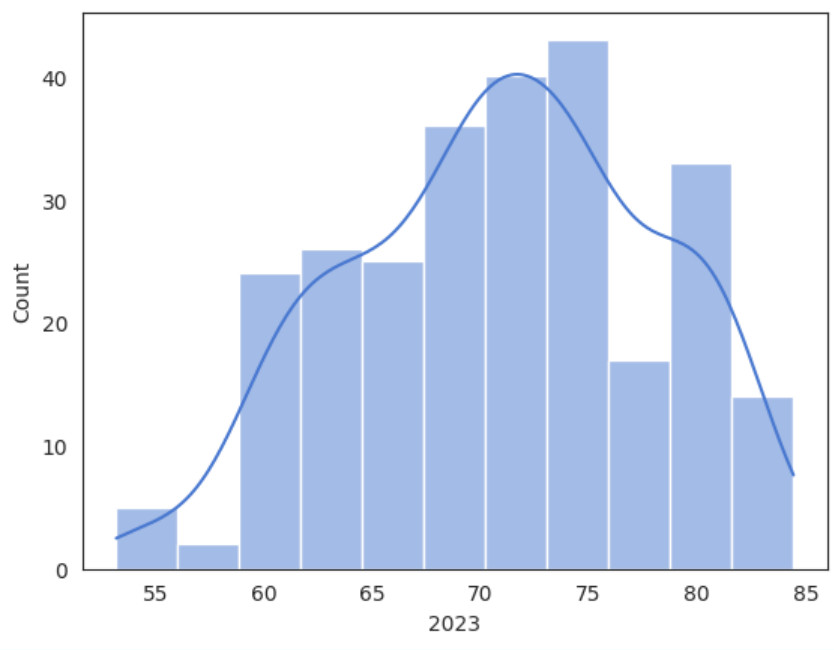
- `cumulative` (aplica a las tres plataformas): Si es `True`, cada *bin* incluye su conteo y el de todos los *bins* para valores más pequeños.

```
lifexp_male['2023'].plot(kind='hist', cumulative=True)
```

Se empleó la paleta **Set2**: `sns.set_palette('Set2')`

- **kde**: Seaborn cuenta con un parámetro `kde` que, al establecerse en `True`, dibuja una estimación de la densidad kernel sobre el histograma, ofreciendo así una representación suavizada de la distribución de los datos.



Aunque matplotlib y pandas no ofrecen una opción directa para agregar un KDE, es posible utilizar seaborn para incorporar fácilmente esta estimación en sus gráficos. Así:

```
plt.hist(lifexp_male['2023'], alpha=0.6, density=True)  
sns.kdeplot(lifexp_male['2023'])
```

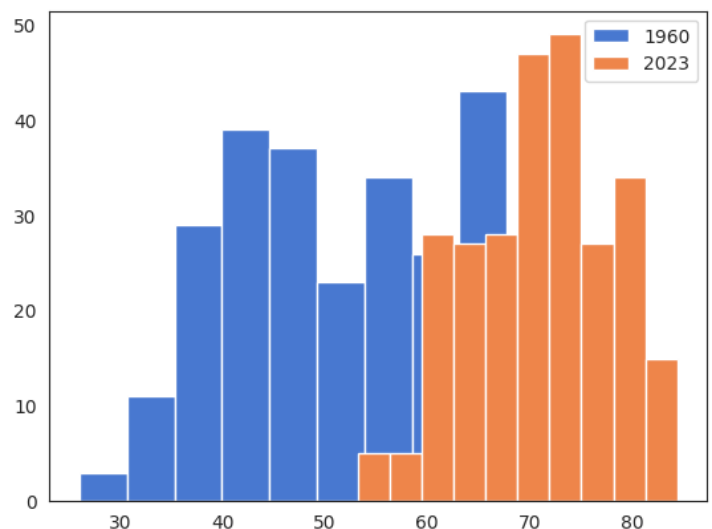
De la instrucción anterior, podrás observar que se ha utilizado el parámetro `density=True`. Esto hace que el área total del histograma sea igual a 1, es decir, convierte el histograma en una distribución de probabilidad.

Si no utilizas `density`, estarías comparando cosas que están en escalas diferentes: una cuenta de valores vs. una función de densidad. Seaborn hace este cálculo de manera interna y añade la curva KDE sobre el histograma.

También puedes colocar varios histogramas en la misma figura, generados de dos o más columnas de tu *dataframe*, o incluso de *dataframes* diferentes. Por ejemplo, comparemos el índice de esperanza de vida de 1960 y 2023, así podrás apreciar los cambios en estos más de 60 años.

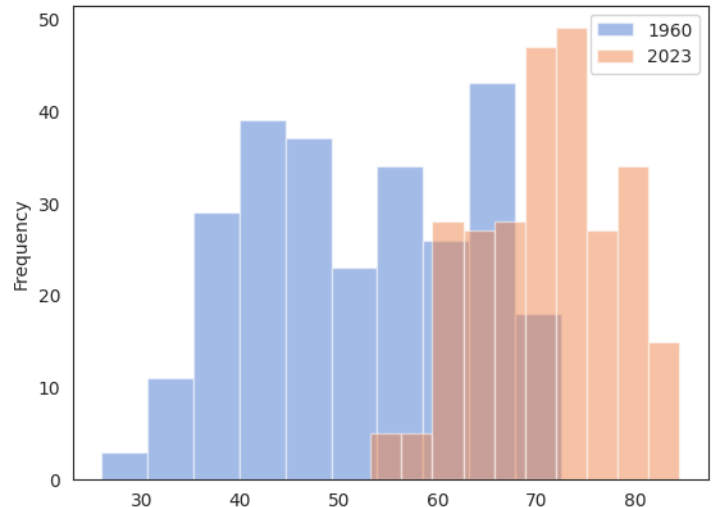
Con matplotlib

```
plt.  
hist('1960', data=lifexp_  
male, label='1960')  
plt.  
hist('2023', data=lifexp_  
male, label='2023')  
plt.legend()
```



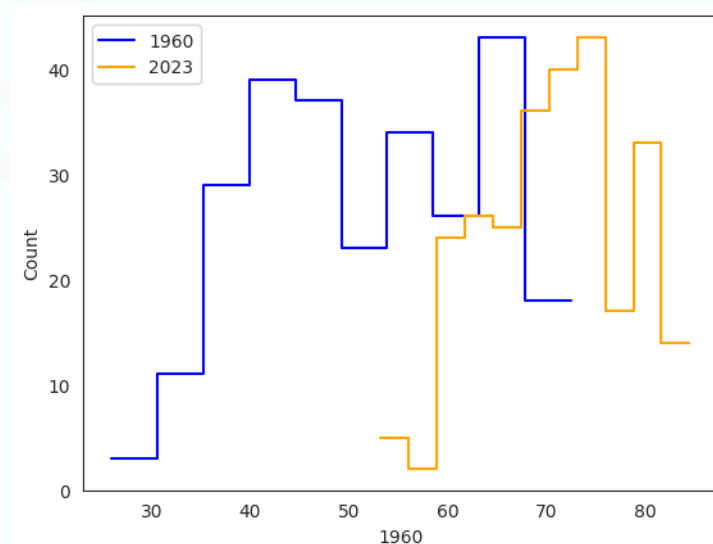
Con pandas

```
lifexp_male['1960'].plot.  
hist(label='1960',alpha=0.5)  
lifexp_male['2023'].plot.  
hist(label='2023',alpha=0.5)  
plt.legend()
```



Con seaborn

```
sns.histplot(x='1960',data=lifexp_  
male,color='blue',label='1960',element='step',fill=False)  
sns.histplot(x='2023',data=lifexp_  
male,color='orange',label='2023',element='step',fill=False)  
plt.legend()
```



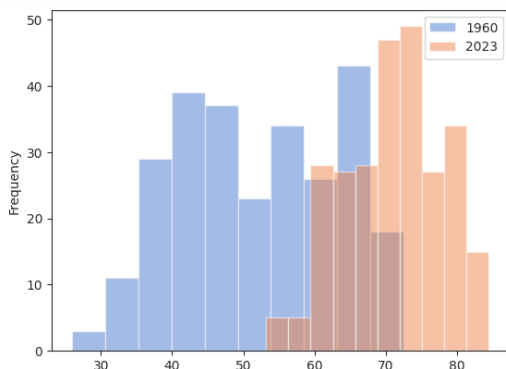
Existen varias configuraciones de los gráficos anteriores que es importante resaltar:

- El parámetro `alpha`, incluido en `pandas`, especifica la transparencia (0 - transparente, 1 - opaco). Por lo tanto, cuando lo establecemos en 0.5 para ambos histogramas, el área sobrepuesta mostrará el color fusionado. En las tres plataformas se utiliza de la misma forma.
- Para mostrar únicamente las líneas en `seaborn`, se emplean los parámetros `element` y `fill` con los valores indicados. Lo equivalente se lograría en `matplotlib` y `pandas` con `histtype='step'`.
- En los tres casos, cada columna tiene sus propios *bins* (10 para 1960 y 10 para 2023).

Este último punto es importante, ya que, **cuando se grafican dos o más columnas en la misma función de trazado, se agrupan los valores de todas las columnas en únicamente 10 bins (o el número de bins que se haya especificado)**. Veamos la diferencia con código de `pandas`:

Graficando las columnas en funciones de trazado independientes

```
lifexp_male['1960'].plot.  
hist(label='1960',alpha=0.5)  
lifexp_male['2023'].plot.  
hist(label='2023',alpha=0.5)
```

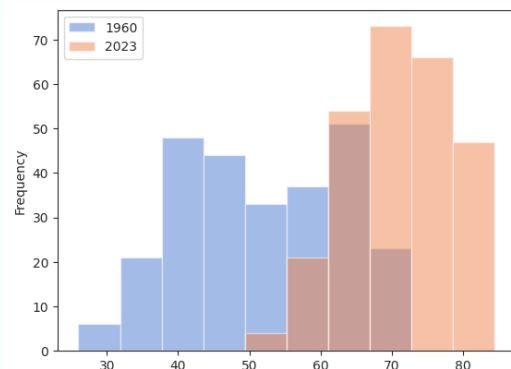


Cada columna tiene sus propios 10 *bins*.

Se empleó el estilo *ticks*: `sns.set_style('ticks')`

Graficando las columnas en la misma función de trazado

```
lifexp_male[['1960','2023']].  
plot.hist(alpha=0.5)
```



Se han creado 10 *bins* considerando los valores de ambas columnas.

Finalmente, veamos cómo graficar una única columna con grupos separados. Para ello, se emplea otra columna que actúa como variable semántica para establecer las separaciones. Por ejemplo, si combinas el dataframe `lifexp_male` con el contenido del archivo **Metadata.csv** (localizado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente), leído en un *dataframe* con el mismo nombre (`metadata`), cada país en el indicador estaría asociado a una región (`Region`) y a un grupo de ingresos (`IncomeGroup`).

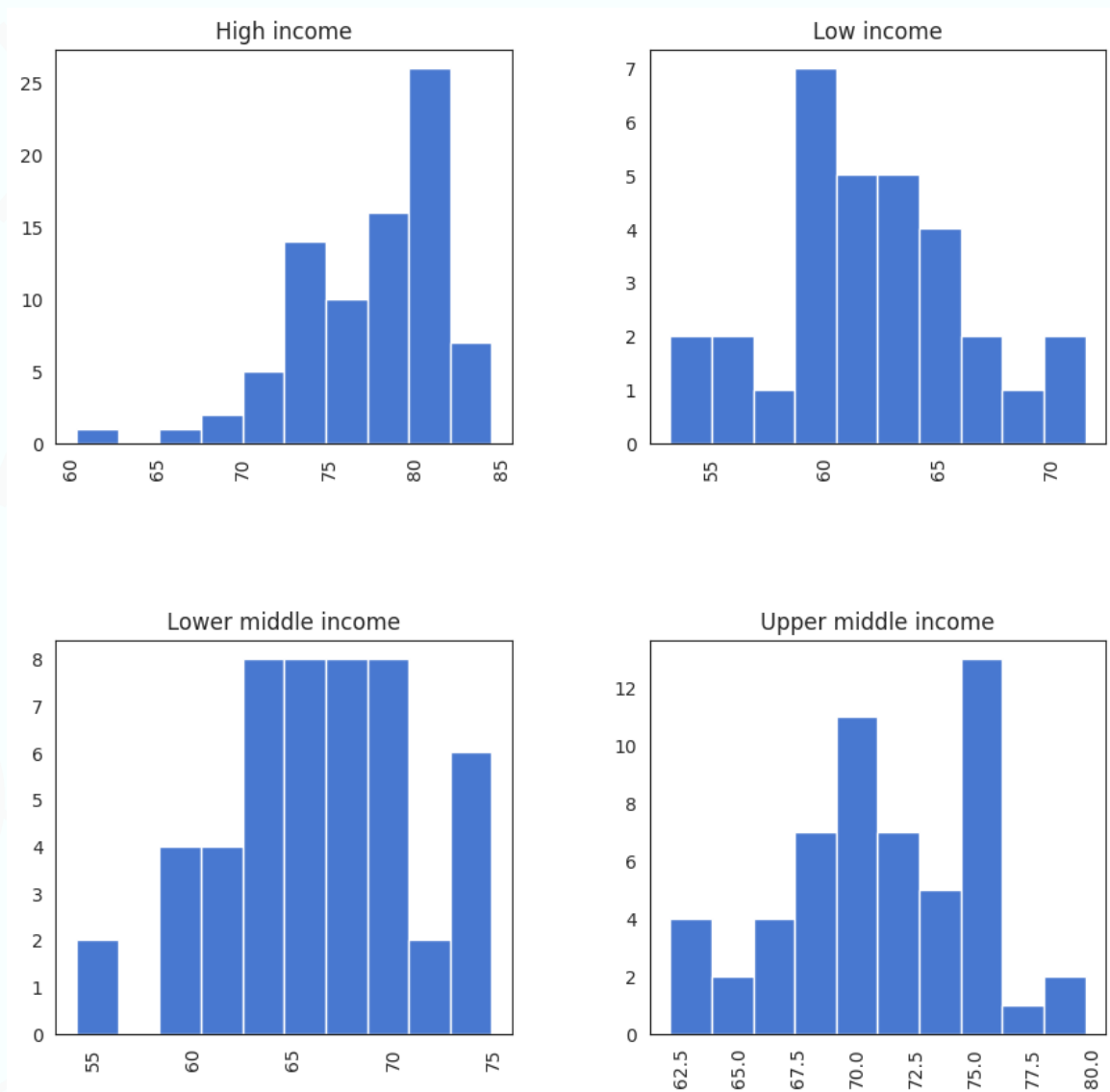
```
merged_df = lifexp_male.merge(metadata,on='Country Code')
```

	Country Name	Country Code	1960	1961	1962	...	2021	2022	2023	Region	IncomeGroup
0	Aruba	ABW	60.746000	61.114000	61.371000	...	70.794000	73.537000	73.702000	Latin America & Caribbean	High income
1	Africa Eastern and Southern	AFE	42.448496	42.681453	43.117691	...	60.351874	61.765707	62.395547	NaN	Low income
2	Afghanistan	AFG	32.136000	32.626000	33.098000	...	57.052000	63.941000	64.467000	South Asia	Low income
3	Africa Western and Central	AFW	35.925673	36.181219	37.031000	...	56.296796	56.906135	57.752356	NaN	Low income
...	...	...	...	...	...	...	...	...	...	...	...
261	Yemen, Rep.	YEM	31.105000	31.720000	30.986000	...	63.090000	65.665000	67.235000	Middle East & North Africa	Low income
262	South Africa	ZAF	50.823000	51.188000	51.707000	...	59.193000	61.813000	62.609000	Sub-Saharan Africa	Upper middle income
263	Zambia	ZMB	49.581000	49.998000	50.258000	...	59.790000	62.768000	63.936000	Sub-Saharan Africa	Lower middle income
264	Zimbabwe	ZWE	51.077000	51.516000	51.965000	...	57.717000	59.880000	60.226000	Sub-Saharan Africa	Lower middle income

265 rows × 68 columns

Del *dataframe* resultante, sería interesante conocer la distribución del indicador por grupo de ingresos. Un modo de hacerlo es a través de la función `hist()` de `pandas`, que crea histogramas de las columnas numéricas del *dataframe*, incluso separadas por una variable de agrupación o semántica. Esta función es diferente a la que hemos utilizado hasta ahora (`plot(kind='hist')` o `plot.hist()`). Veamos cómo:

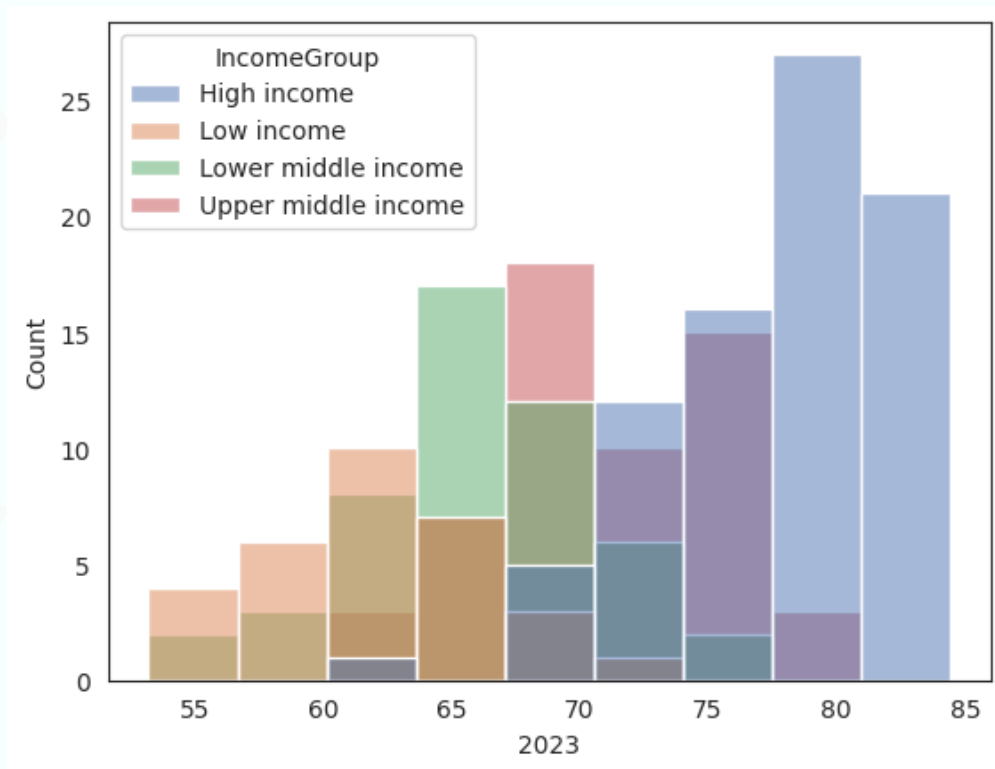
```
merged_df['2023'].hist(by=merged_df['IncomeGroup'],figsize=(10,10))
```



La función `hist()` permite especificar el parámetro `by` para indicar la variable que separa los histogramas en grupos, `IncomeGroup` en este caso. Como es de esperarse, el grupo de altos ingresos (`High income`) es el que tiene la mayor esperanza de vida.

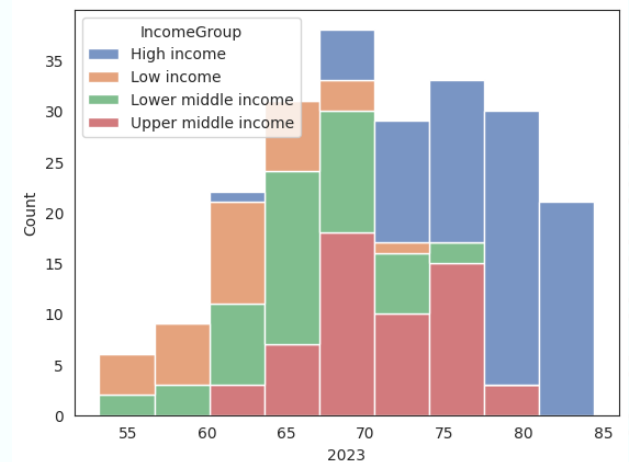
Seaborn también permite dividir en grupos el conteo de valores de una misma columna, haciendo uso del parámetro `hue`. A diferencia de `hist()`, coloca la información de los cuatro grupos en el mismo histograma:

```
sns.histplot(x='2023', data=merged_df, hue='IncomeGroup')
```

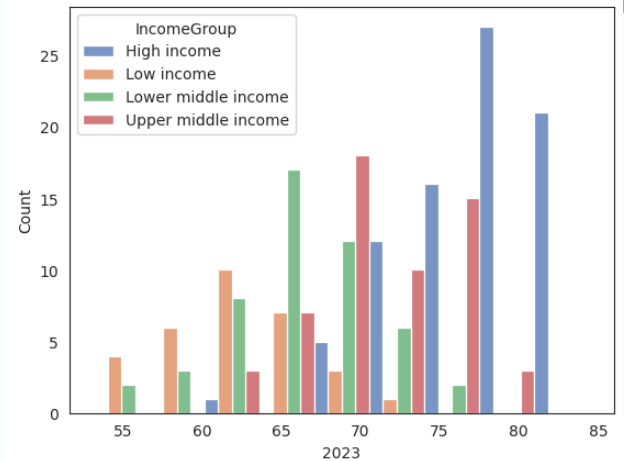


Por defecto, los histogramas se superponen, pero puedes cambiarlo con el parámetro `multiple`:

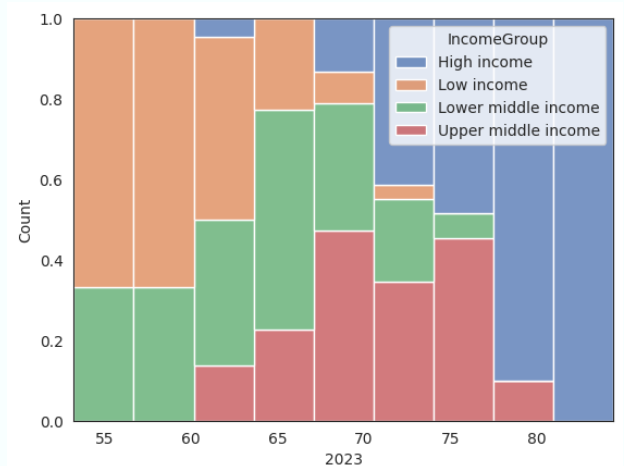
```
sns.  
histplot(x='2023', data=merged_  
df,  
hue='IncomeGroup',  
multiple='stack')
```



```
sns.  
histplot(x='2023', data=merged_  
df,  
hue='IncomeGroup',  
multiple='dodge')
```

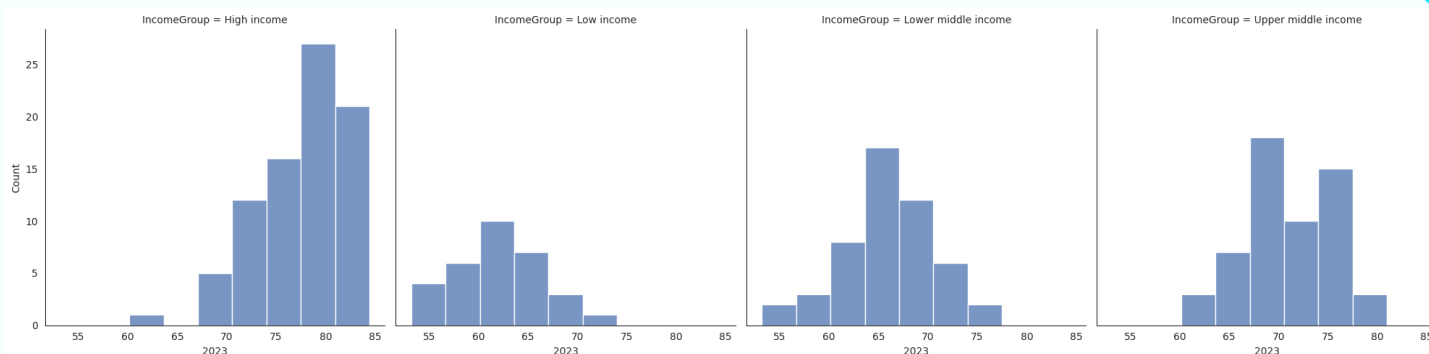


```
sns.  
histplot(x='2023', data=merged_  
df,  
hue='IncomeGroup',  
multiple='fill')
```



Con seaborn también es posible separar las agrupaciones semánticas en gráficos independientes, empleando la interfaz `displot` e indicándolo en el parámetro `col`:

```
sns.  
displot(kind='hist', x='2023', col='IncomeGroup', data=merged_  
df)
```

Como has visto en este apartado, los histogramas son herramientas fundamentales para comprender la distribución de los datos, permitiendo visualizar de manera clara la frecuencia de ocurrencia de diferentes rangos de valores. Al ser una de las primeras herramientas empleadas en el análisis exploratorio de datos, los histogramas no solo son esenciales para el análisis descriptivo, sino también para la toma de decisiones informadas en diversos contextos, como la estadística, la investigación científica o el análisis de datos empresariales.

Antes de terminar...

Aunque los histogramas y los gráficos de barras muestran la frecuencia de los valores representados, los primeros se emplean cuando los datos son cuantitativos continuos; es decir, en el eje X no se presentan categorías, sino rangos de valores denominados clases o *bins*. Puedes diferenciar entre un tipo de gráfico y otro fijándote si hay o no separación entre las barras.

Piensa en las bases de datos que manipulas cada día:

- ¿Tienes idea de cómo están distribuidos los valores de cada atributo o columna?
- ¿Con qué frecuencia se presentan ciertos fenómenos (valores) relevantes para tu organización?, ¿cuáles hay que minimizar y cuáles maximizar a raíz de este descubrimiento?

Pruébate

Te invitamos a analizar las siguientes preguntas, así como sus respectivas respuestas.

- 1. Imagina que te han solicitado hacer un estudio comparativo de los indicadores de esperanza de vida por género. La ONG donde colaboras posee dos archivos con información al respecto: **LifeExpectancyFemale.csv** y **LifeExpectancyMale.csv** (localizados en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente), mismos que han sido almacenados en los *dataframes* `lifexp_female` y `lifexp_male`, respectivamente.

lifexp_female

	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
0	Aruba	ABW	67.459000	67.394000	67.922000	...	78.814000	78.333000	76.417000	78.691000	78.779000
1	Africa Eastern and Southern	AFE	45.997686	46.368461	46.743936	...	66.434436	66.536672	65.652968	67.231130	67.914779
2	Afghanistan	AFG	33.549000	34.043000	34.502000	...	66.144000	64.656000	64.079000	67.236000	67.536000
3	Africa Western and Central	AFW	39.817586	40.120915	40.411312	...	58.204817	58.553686	58.473634	59.143474	59.997102
...	...	...	...	...	...	...	...	...	...	...	...
261	Yemen, Rep.	YEM	36.028000	36.592000	36.670000	...	70.061000	69.805000	69.170000	70.319000	71.388000
262	South Africa	ZAF	54.578000	55.218000	55.679000	...	69.216000	67.948000	64.772000	69.037000	69.599000
263	Zambia	ZMB	51.708000	52.075000	52.393000	...	65.440000	65.953000	64.925000	67.717000	68.673000
264	Zimbabwe	ZWE	55.963000	56.470000	56.993000	...	63.049000	63.747000	62.289000	64.544000	65.013000

265 rows x 66 columns

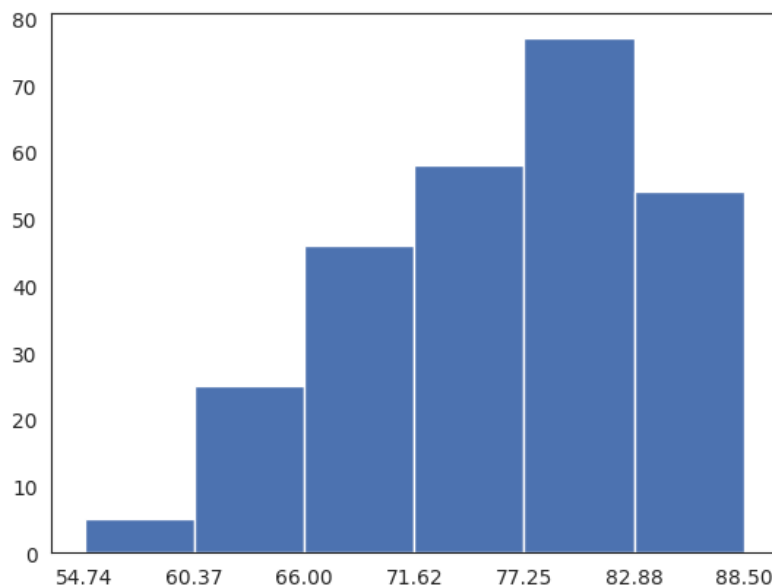
lifexp_male

	Country Name	Country Code	1960	1961	1962	...	2019	2020	2021	2022	2023
0	Aruba	ABW	60.746000	61.114000	61.371000	...	73.047000	72.344000	70.794000	73.537000	73.702000
1	Africa Eastern and Southern	AFE	42.448496	42.681453	43.117691	...	61.289264	61.048093	60.351874	61.765707	62.395547
2	Afghanistan	AFG	32.136000	32.626000	33.098000	...	59.910000	58.455000	57.052000	63.941000	64.467000
3	Africa Western and Central	AFW	35.925673	36.181219	37.031000	...	56.145037	56.219168	56.296796	56.906135	57.752356
...	...	...	...	...	...	...	...	...	...	...	...
261	Yemen, Rep.	YEM	31.105000	31.720000	30.986000	...	63.337000	63.330000	63.090000	65.665000	67.235000
262	South Africa	ZAF	50.823000	51.188000	51.707000	...	62.696000	62.174000	59.193000	61.813000	62.609000
263	Zambia	ZMB	49.581000	49.998000	50.258000	...	60.349000	60.731000	59.790000	62.768000	63.936000
264	Zimbabwe	ZWE	51.077000	51.516000	51.965000	...	58.830000	59.091000	57.717000	59.880000	60.226000

265 rows x 66 columns

Estás considerando analizar la distribución de los datos del 2023, utilizando seis clases (o *bins*). Si cuentas con la siguiente información del *dataframe* `lifexp_female`, ¿de qué tamaño serían sus *bins* y qué afirmación respecto a la expectativa de vida es correcta?

```
lifexp_female['2023'].min()  
54.71lifexp_female['2023'].max()  
88.5
```

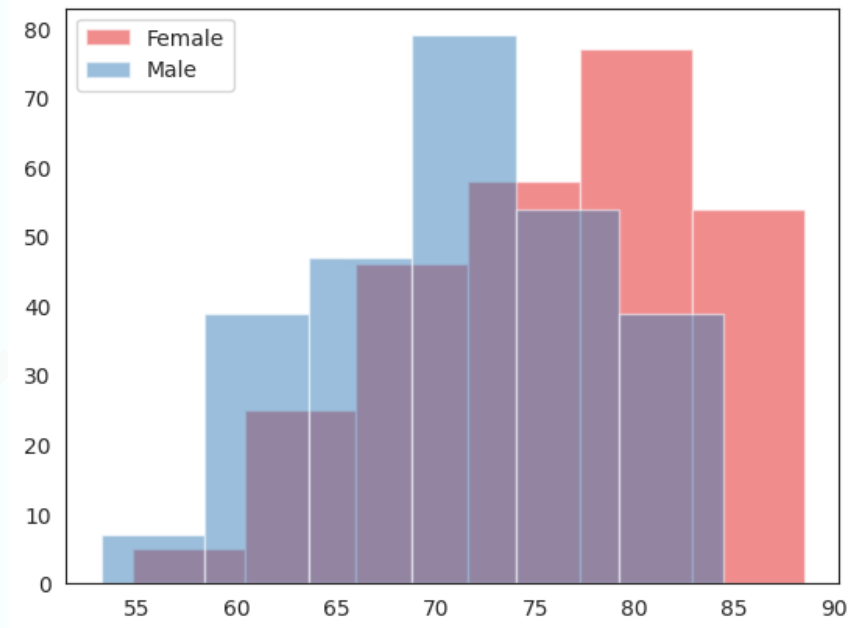


- a. Sus *bins* son de 5.6 años y hay cinco países con expectativa de cinco años.
- b. Sus *bins* son de 10 años y la mayoría de los países tienen expectativa de 70 años.
- c. Sus *bins* son de 5.6 años y el rango más frecuente está entre los 77 y 83 años.

Retroalimentación

El tamaño de los *bins* se calcula como $(\text{max} - \text{min}) / 6$ y el rango más frecuente corresponde a la barra mayor en el diagrama.

2. Si quisieras comparar la distribución de ambos sexos en un mismo diagrama, utilizando la plataforma matplotlib, y que cada histograma tenga sus seis *bins* independientes, ¿qué código deberías utilizar?



- a. `lifexp_female['2023'].
hist(bins=6,alpha=0.5,label='Female')
lifexp_male['2023'].hist(bins=6,alpha=0.5,label='Male')
plt.legend()`
- b. `plt.hist('2023',data=lifexp_
female,bins=6,alpha=0.5,label='Female')
plt.hist(lifexp_
male['2023'],bins=6,alpha=0.5,label='Male')
plt.legend()`
- c. `plt.hist([lifexp_female['2023'],lifexp_
male['2023']],bins=6,label=['Female','Male'])
plt.legend()`

Retroalimentación

En el código se generan dos histogramas con seis *bins* independientes.

3. Imagina que has combinado en un único *dataframe* (`lifexp`) la información del 2023 de hombres y mujeres, así como el continente al que pertenece cada país, como se muestra en la imagen:

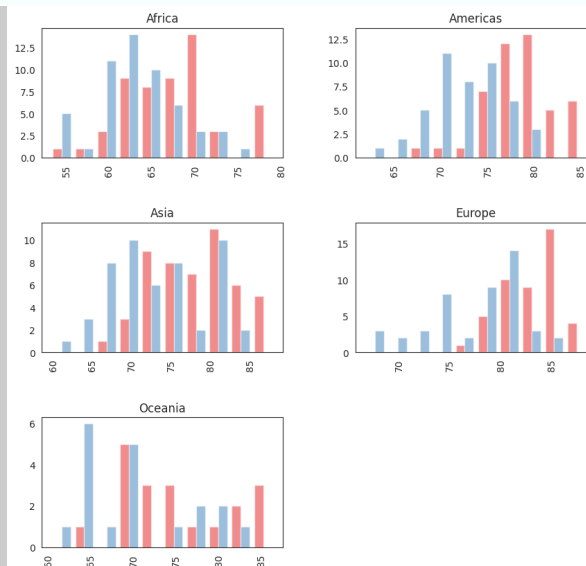
	Country Code	female	male	Name	Continent	Region
0	ABW	78.779	73.702	Aruba	Americas	Latin America and the Caribbean
1	AFG	67.536	64.467	Afghanistan	Asia	Southern Asia
2	AGO	67.144	62.099	Angola	Africa	Sub-Saharan Africa
3	ALB	81.446	77.728	Albania	Europe	Southern Europe
...	...	...	...	...	...	...
211	YEM	71.388	67.235	Yemen	Asia	Western Asia
212	ZAF	69.599	62.609	South Africa	Africa	Sub-Saharan Africa
213	ZMB	68.673	63.936	Zambia	Africa	Sub-Saharan Africa
214	ZWE	65.013	60.226	Zimbabwe	Africa	Sub-Saharan Africa

215 rows × 6 columns

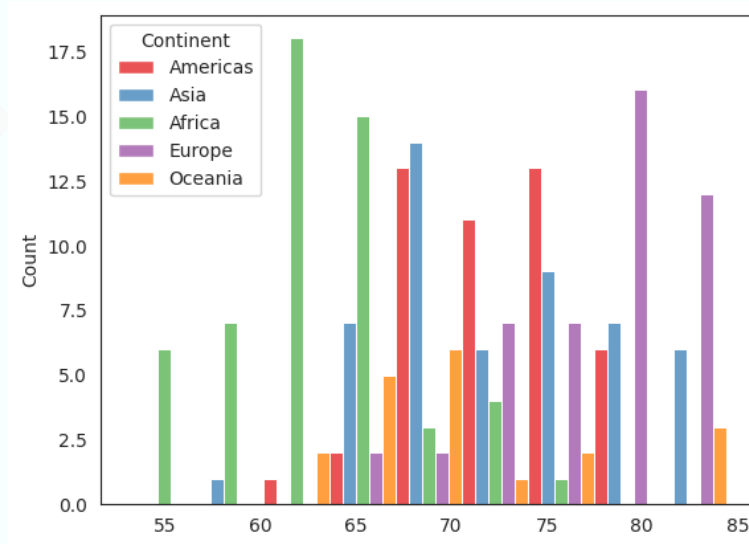
¿Qué gráfico genera el siguiente código?

```
lifexp[['female','male']].  
hist(by=lifexp['Continent'],alpha=0.5,figsize=(10,10))
```

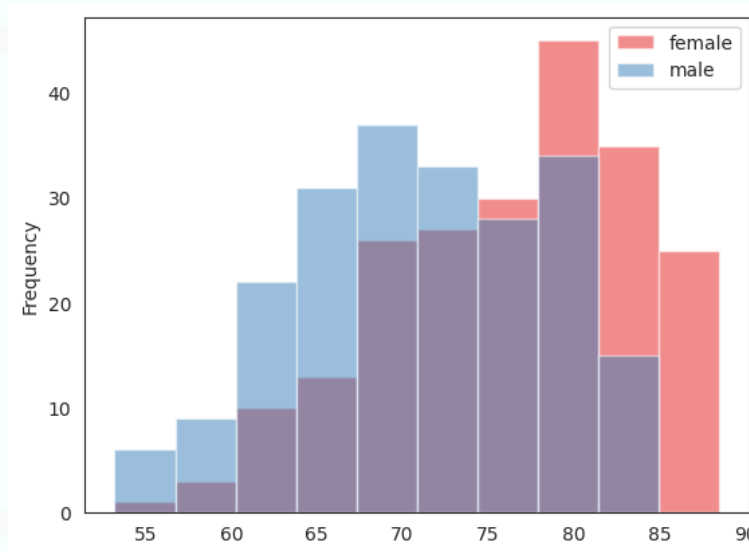
a.



b.



c.



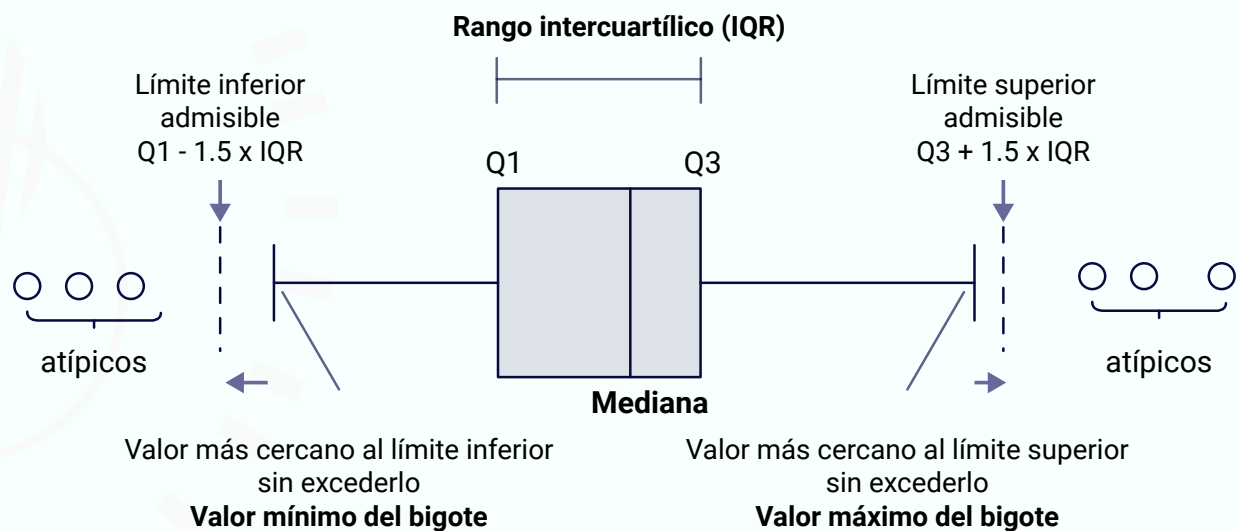
Retroalimentación

En el código se generan dos histogramas con 6 *bins* independientes.

Diagramas de caja y bigote (*boxplots*)

Los diagramas de caja y bigote (*boxplot*) se utilizan para mostrar la distribución de datos cuantitativos continuos con relación a una variable categórica; ofrecen un panorama de la distribución de dichos valores, a través de sus cuartiles. Para ello, utilizan como representación una caja y segmentos (bigotes) que delimitan los intervalos donde la variable continua concentra la mayoría de las observaciones. En la caja se ubican los cuartiles; en los márgenes de los bigotes inferior y superior y fuera de estos, los valores atípicos (*outliers*) que se especifican mediante puntos.

Utiliza este tipo de diagrama cuando desees evaluar la distribución y la tendencia central de un conjunto de datos.



Recuerda que los cuartiles son valores que dividen un conjunto de datos ordenado en cuatro partes iguales, y ayudan a entender cómo se distribuyen los datos. Hay tres cuartiles principales:

1. Primer cuartil (Q_1): Es el valor que separa el 25 % más bajo del resto de los datos, es decir, el 25 % de los datos están por debajo de Q_1 .
2. Segundo cuartil (Q_2): Es la mediana; el valor que divide los datos en dos mitades. El 50 % de los datos están por debajo de este valor.
3. Tercer cuartil (Q_3): Marca el punto debajo del cual se encuentra el 75 % de los datos.

Con estos valores también se puede calcular el rango intercuartílico (IQR), que es la diferencia entre Q_3 y Q_1 , y se utiliza para identificar la dispersión de los datos centrales y detectar valores atípicos.

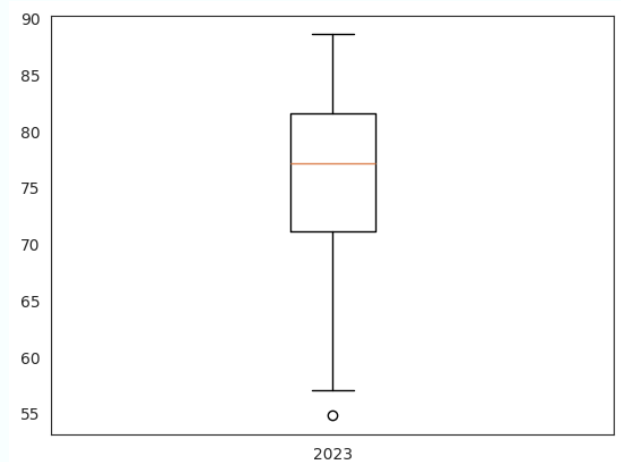
Para ejemplificar su uso, utilizaremos el *dataframe* `lifexp_female` que concentra la información del archivo **LifeExpectancyFemale.csv** (localizado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente), y graficaremos un *boxplot* con la expectativa de vida del año 2023, empleando varias de las formas equivalentes estudiadas.

Con matplotlib

```
plt.boxplot(lifexp_female['2023'], tick_labels=['2023'])
```

O

```
plt.boxplot('2023', data=lifexp_female, tick_labels=['2023'])
```

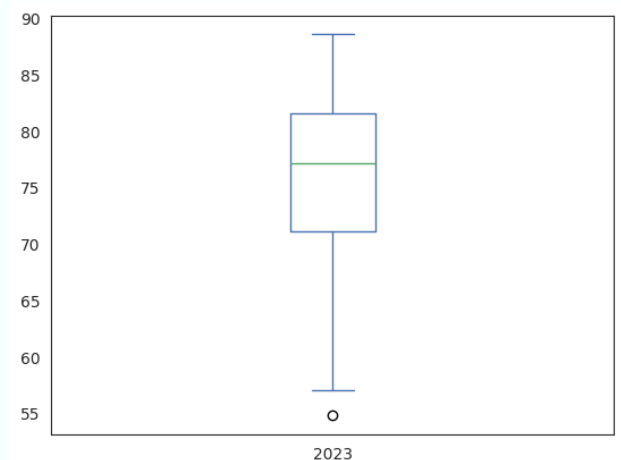


Con pandas

```
lifexp_female['2023'].plot.box()
```

O

```
lifexp_female['2023'].plot(kind='box')
```



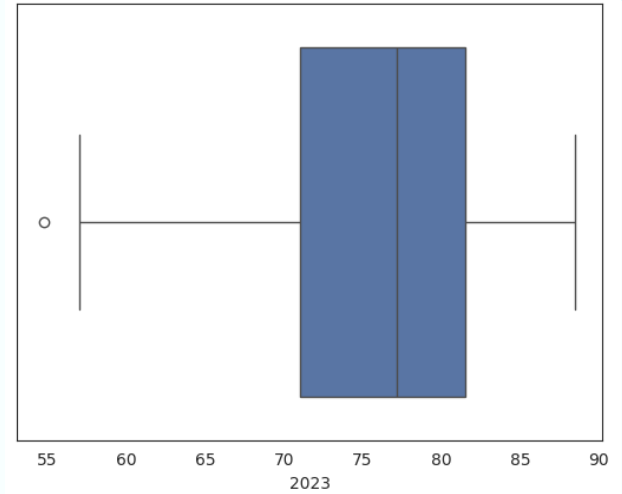
En ambos casos, para girar el diagrama emplea el parámetro `vert=False`.

Con seaborn

```
sns.boxplot(x=lifexp_
female['2023'])
```

o

```
sns.
boxplot(x='2023', data=lifexp_
female)
```



Para girar el diagrama emplea el parámetro `orient='h'` y cambia la variable **X** por **Y**.

En las figuras anteriores se muestran los tres cuartiles en la caja: 25 % o 0.25 (1^{er} cuartil), 50 % o 0.50 (2° cuartil - mediana) y 75 % o 0.75 (3^{er} cuartil). Si obtienes los valores específicos de los *boxplots* previos, entonces:

- 1^{er} cuartil (Q1)

```
Q1 = lifexp_female['2023'].quantile(.25)
```

Lo que significa que el 25% de los países tiene expectativa de vida menor o igual a los 71 años.

- 2° cuartil = mediana (Q2)

```
Q2 = lifexp_female['2023'].quantile(.5) = lifexp_
female['2023'].median()
77.134
```

Lo que significa que el 50 % de los países tiene expectativa de vida menor o igual a los 77 años.

- 3^{er} cuartil (Q3)

```
Q3 = lifexp_female['2023'].quantile(.75)
81.5
```

Lo que significa que el 75 % de los países tiene expectativa de vida menor o igual a los 81.5 años.

- Rango intercuartílico

Los segmentos o bigotes son calculados en función del rango intercuartílico, que mide la distancia entre el primer y tercer cuartil ($Q3 - Q1$); de esta manera, abarca el 50 % central de los datos.

```
IQR = Q3 - Q1
10.448
```

- Límites de los bigotes

A partir del rango intercuartílico se calculan los límites admisibles inferior (LÍMITE_INF) y superior (LÍMITE_SUP).

```
LÍMITE_INF = Q1 - 1.5 IQR
55.38
LÍMITE_SUP = Q3 + 1.5 IQR
97.172
```

Los valores mínimo y máximo de los bigotes (BIGOTE_MIN y BIGOTE_MAX) corresponden a los datos más cercanos a los límites admisibles calculados, sin excederlos.

```
BIGOTE_MIN = menor elemento del conjunto >= LIMITE_INF
BIGOTE_MIN = min(x[x >= LIMITE_INF])
BIGOTE_MIN = min(lifexp_female['2023'][lifexp_female['2023']
>= LIMITE_INF])
57.014
BIGOTE_MAX = mayor elemento del conjunto <= LIMITE_SUP
BIGOTE_MAX = max(x[x <= LIMITE_SUP])

BIGOTE_MAX = max(lifexp_female['2023'][lifexp_female['2023']
<= LIMITE_SUP])88.504
```

El bigote inferior se dibujará entonces desde el valor mínimo del bigote hasta el primer cuartil:

```
[BIGOTE_MIN - Q1] [57.014-71.052]
```

El bigote superior se dibujará desde el tercer cuartil hasta el valor máximo del bigote:

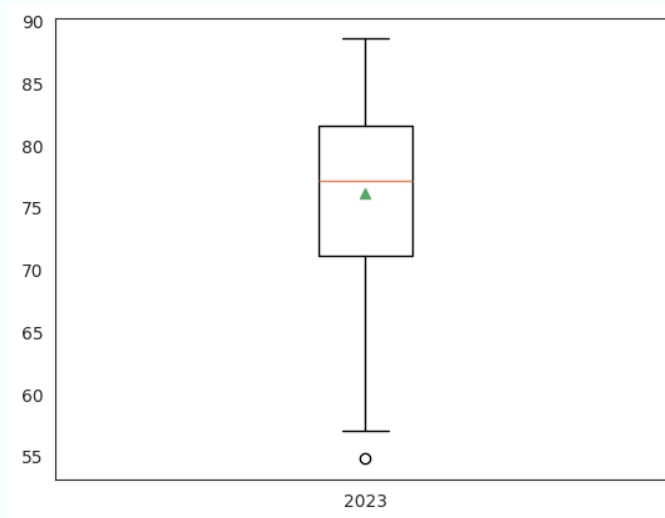
```
[Q3 - BIGOTE_MAX] [81.5-88.504]
```

Todos los valores menores que `BIGOTE_MIN` o mayores que `BIGOTE_MAX` se denominan **valores atípicos** (*outliers*) y son representados como puntos o círculos.

Ahora bien, algunos parámetros de utilidad que aplican a las tres plataformas son:

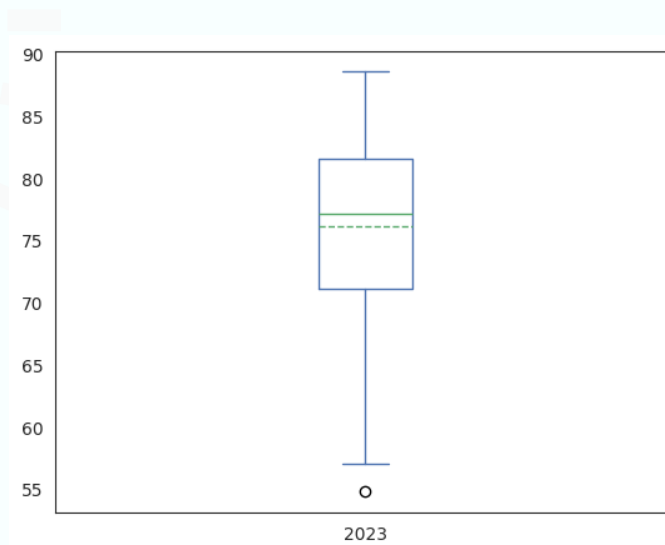
- `showmeans`: Si es `True`, muestra la media como un punto o símbolo.

```
plt.boxplot(lifexp_female['2023'], tick_
labels=['2023'], showmeans=True)
```



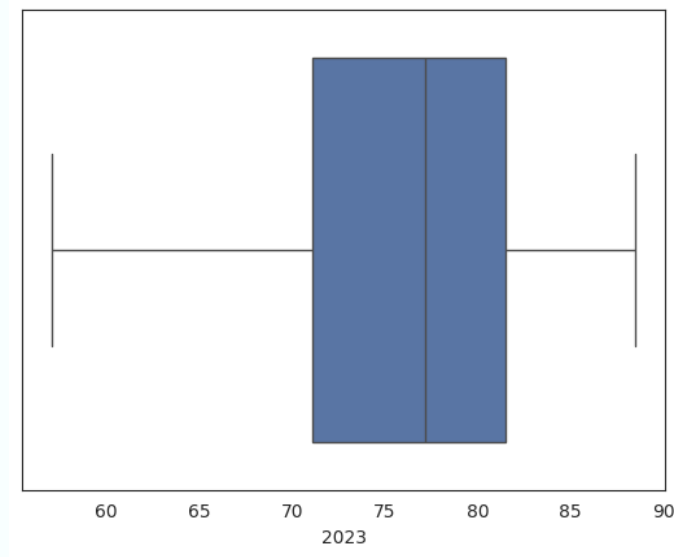
- **meanline:** Si la opción de `showmeans` está activada, este parámetro cambia el símbolo de la media por una línea. Útil para comparar media vs. mediana.

```
lifexp_female['2023'].plot.box(showmeans=True, meanline=True)
```



- **showfliers:** Controla si se muestran los valores atípicos. Si es `False`, los omite.

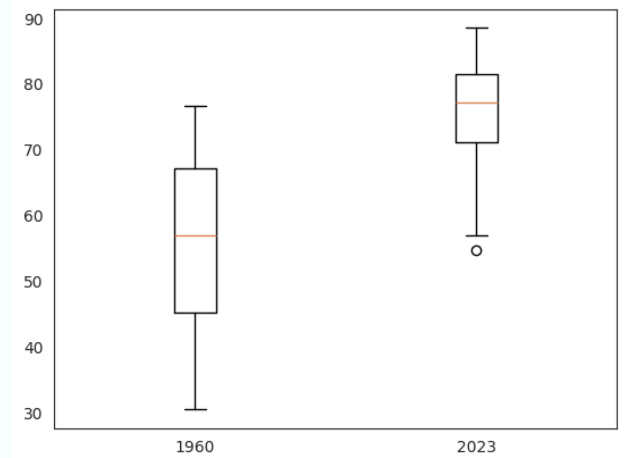
```
sns.boxplot(x=lifexp_female['2023'], showfliers=False)
```



También puedes colocar varios *boxplot* en la misma figura, generados de dos o más columnas de tu *dataframe*, o incluso de *dataframes* diferentes. Por ejemplo, comparemos el índice de esperanza de vida de 1960 y 2023, así podrás apreciar los cambios en estos casi más de 60 años.

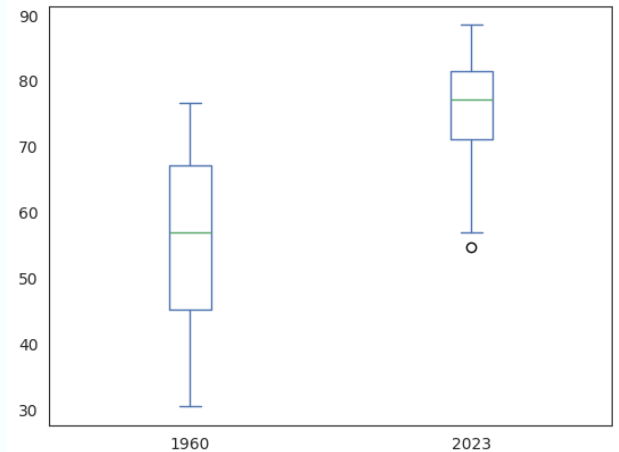
Con matplotlib

```
plt.boxplot(x=[lifexp_
female['1960'].dropna(),lifexp_
female['2023']],tick_
labels=['1960','2023'])
```



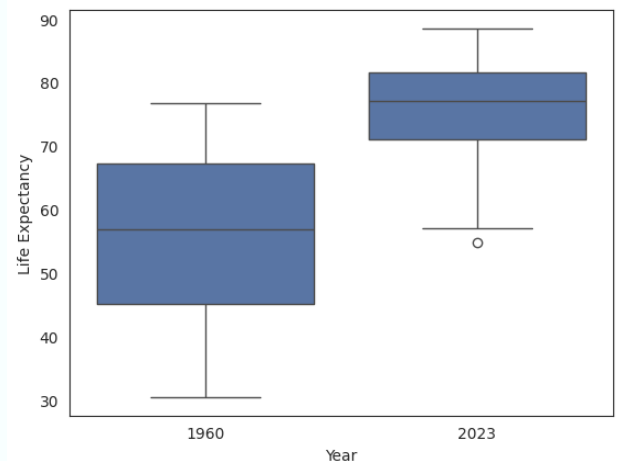
Con pandas

```
lifexp_male[['1960', '2019']].  
plot.box()
```



Con seaborn

```
long = lifexp_female.  
loc[:, ['1960', '2023']].melt(var_  
name='Year', value_name='Life  
Expectancy')  
sns.  
boxplot(data=long, x='Year', y='Life  
Expectancy')
```



Existen varias cuestiones que debes observar de los gráficos anteriores:

- En matplotlib, es importante eliminar los valores vacíos con `dropna()` antes de graficar. A diferencia de otras plataformas que manejan los valores faltantes por sí solas, en matplotlib la presencia de `NaN` puede hacer que el *boxplot* no se dibuje. No se genera un error explícito, simplemente no aparece la gráfica. En seaborn, primero es necesario reestructurar el *dataframe* a formato largo (también conocido como **tidy data**), ya que seaborn trabaja mejor con datos en ese formato. Cada fila representa una observación individual, y hay columnas separadas para la variable de agrupación semántica (en este caso, el año) y la variable numérica que se quiere graficar (la esperanza de vida).

Antes

```
lifexp_female.  
loc[:, ['1960', '2023']]
```



	1960	2023
0	67.459000	78.779000
1	45.997686	67.914779
2	33.549000	67.536000
3	39.817586	59.997102
...	...	...
261	36.028000	71.388000
262	54.578000	69.599000
263	51.708000	68.673000
264	55.963000	65.013000

265 rows × 2 columns

Después

```
lifexp_female.  
loc[:, ['1960', '2023']].  
melt(var_name='Year', value_  
name='Life Expectancy')
```



	Year	Life Expectancy
0	1960	67.459000
1	1960	45.997686
2	1960	33.549000
3	1960	39.817586
...	...	...
526	2023	71.388000
527	2023	69.599000
528	2023	68.673000
529	2023	65.013000

530 rows × 2 columns

Como en los histogramas, en los *boxplots* puedes graficar una única columna y emplear otra para separarla en grupos. Retomando el *dataframe* *metadata*, que combina el indicador de expectativa de vida por país, con la región y el grupo de ingresos, podrías obtener un *boxplot* por región, de la siguiente manera:

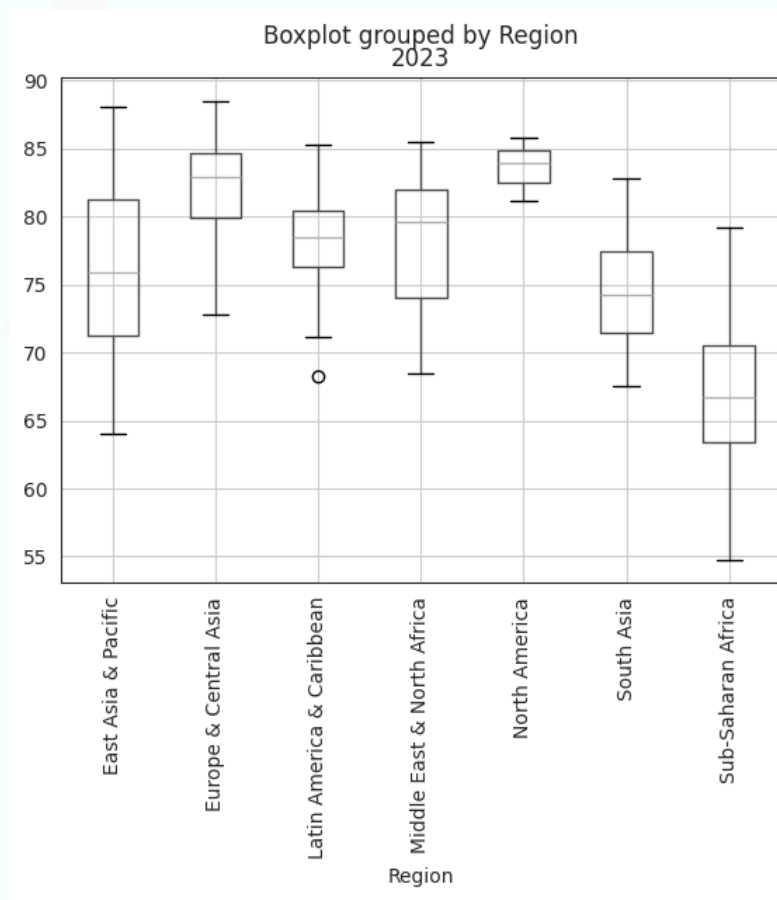
```
merged_df = lifexp_male.merge(metadata, on='Country Code')
```

	Country Name	Country Code	1960	1961	1962	...	2021	2022	2023	Region	IncomeGroup
0	Aruba	ABW	67.459000	67.394000	67.922000	...	76.417000	78.691000	78.779000	Latin America & Caribbean	High income
1	Africa Eastern and Southern	AFE	45.997686	46.368461	46.743936	...	65.652968	67.231130	67.914779	NaN	Low income
2	Afghanistan	AFG	33.549000	34.043000	34.502000	...	64.079000	67.236000	67.536000	South Asia	Low income
3	Africa Western and Central	AFW	39.817586	40.120915	40.411312	...	58.473634	59.143474	59.997102	NaN	Low income
...	...	...	...	...	...	...	...	...	...	...	...
261	Yemen, Rep.	YEM	36.028000	36.592000	36.670000	...	69.170000	70.319000	71.388000	Middle East & North Africa	Low income
262	South Africa	ZAF	54.578000	55.218000	55.679000	...	64.772000	69.037000	69.599000	Sub-Saharan Africa	Upper middle income
263	Zambia	ZMB	51.708000	52.075000	52.393000	...	64.925000	67.717000	68.673000	Sub-Saharan Africa	Lower middle income
264	Zimbabwe	ZWE	55.963000	56.470000	56.993000	...	62.289000	64.544000	65.013000	Sub-Saharan Africa	Lower middle income

265 rows x 68 columns

Un modo de hacerlo es a través de la función `boxplot()` de pandas, que permite la separación por una variable de agrupación. Esta función es diferente a la que hemos utilizado hasta ahora (`plot(kind='hist')` o `plot.hist()`). Veamos cómo:

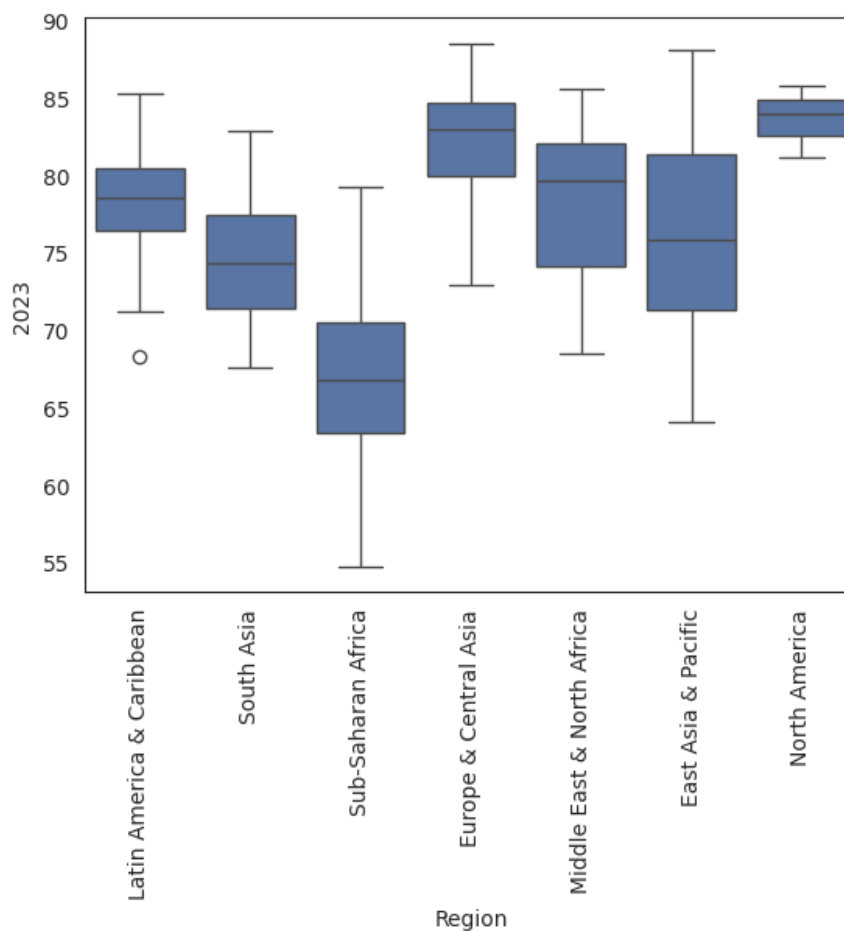
```
merged_df.boxplot(column=['2023'], by='Region', rot=90)
```



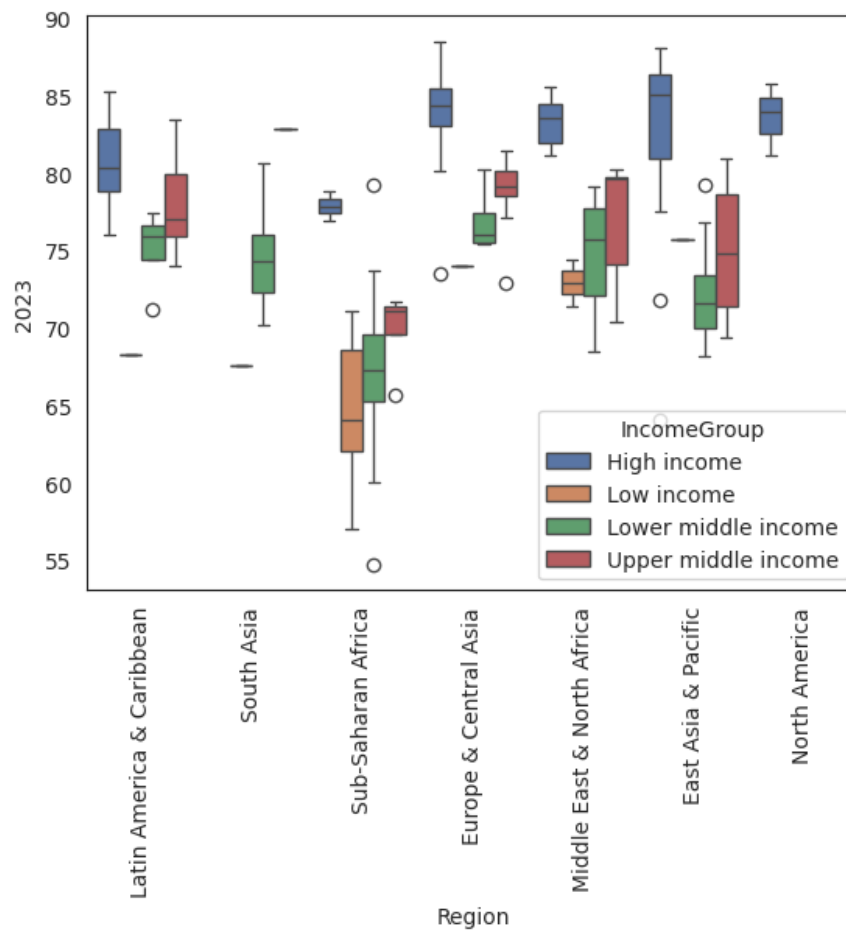
La función `boxplot()` permite especificar el parámetro `by` para indicar la variable que separa en grupos los *boxplots*, `Region` en este caso. De la figura obtenida se observa que `North America` es la región con menor dispersión.

Seaborn también permite dividir los valores de una misma columna en grupos, haciendo uso del parámetro `y`. Con el parámetro `hue` se podría hacer todavía otra división semántica.

```
sns.boxplot(x='Region', y='2023', data=merged_df)
plt.xticks(rotation=90)
```

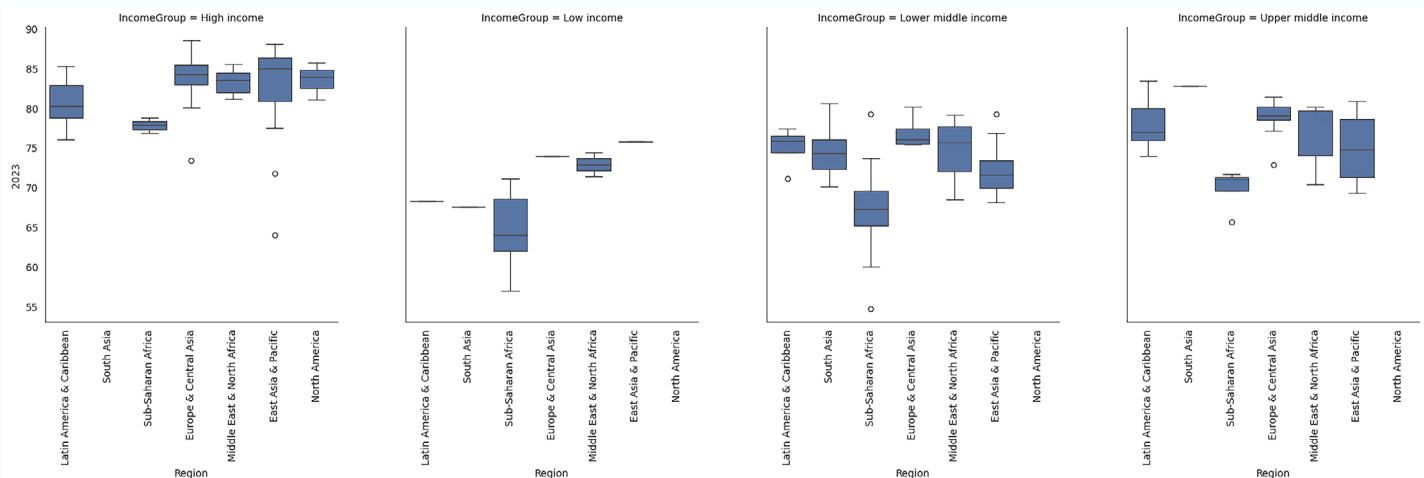


```
sns.  
boxplot(x='Region', y='2023', hue='IncomeGroup', data=merged_  
df)  
plt.xticks(rotation=90)
```



Si el gráfico que incluye dos separaciones semánticas (con los parámetros `y` y `hue`) se muestra muy cargado, tienes la opción de utilizar la interfaz `catplot` para dividirlo, basta con indicarlo en el parámetro `col`:

```
splot = sns.  
catplot(kind='box', x='Region', y='2023', col='IncomeGroup',  
data=merged_df);  
splot.set_xticklabels(rotation=90)
```



Lo anterior hace que los *boxplots* sean especialmente convenientes cuando se comparan distribuciones entre diferentes grupos, ya que facilitan la identificación de diferencias clave en los rangos y dispersión de los datos. Al mostrar la mediana, los cuartiles y los valores atípicos de manera visual, los *boxplots* permiten observar rápidamente si hay grupos con mayor variabilidad o si existen diferencias significativas en la distribución central de los datos.

Antes de terminar...

Los *boxplots* son una herramienta útil y rápida para visualizar el grado de dispersión de tus datos y permiten hacer comparaciones entre grupos o categorías de varias medidas estadísticas con un único diagrama.

Reflexiona nuevamente acerca de la información que utilizas en tu labor profesional:

- ¿Qué tan dispersos se encuentran los valores de cada variable o columna?, ¿en qué casos es conveniente para tu organización y en cuáles no?
- ¿Un análisis con *boxplots* sería un instrumento válido para tomar ciertas decisiones?

Pruébate

Realiza los siguientes ejercicios para reforzar los conocimientos que has adquirido en esta lección.

Sección 1

Descarga el documento “Plantilla” (ubicado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente) y realiza los siguientes pasos:

1. Lee en dos *dataframes*: `adolescent_fertility` y `mortality`, el contenido de los archivos **AdolescentFertility.csv** y **Mortality.csv** (ubicados en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente), respectivamente, que almacenan, por país, las tasas de fecundidad adolescente y mortalidad de menores de 5 años desde 1960.

La tasa de fecundidad adolescente es el número de nacimientos por cada 1.000 mujeres de 15 a 19 años.

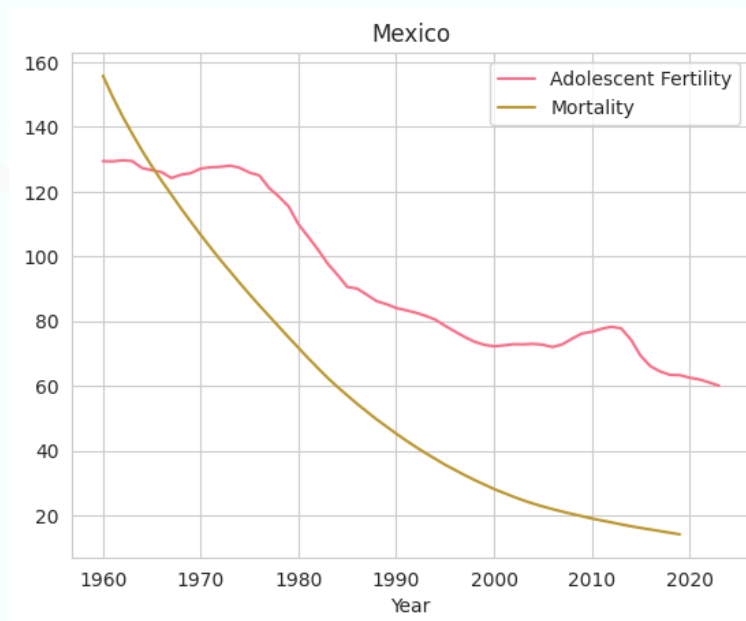
La tasa de mortalidad de menores de cinco años es la probabilidad por cada 1.000 de que un bebé recién nacido muera antes de cumplir los cinco años. La última actualización de este indicador es del 2019.

2. A partir de los *dataframes* anteriores, obtén uno donde únicamente almacenes la información de México. Haz los cambios necesarios para que quede con la siguiente estructura:

Adolescent Fertility Mortality		
Year		
1960	129.434	155.8
1961	129.367	149.3
1962	129.677	143.3
1963	129.470	137.9
...	...	...
2020	62.577	NaN
2021	62.033	NaN
2022	61.111	NaN
2023	60.099	NaN

64 rows x 2 columns

3. Grafica el *dataframe* anterior utilizando la plataforma pandas, para conocer la evolución de estos índices a través de los años. Personaliza las propiedades del gráfico para que se vea como en la siguiente imagen:

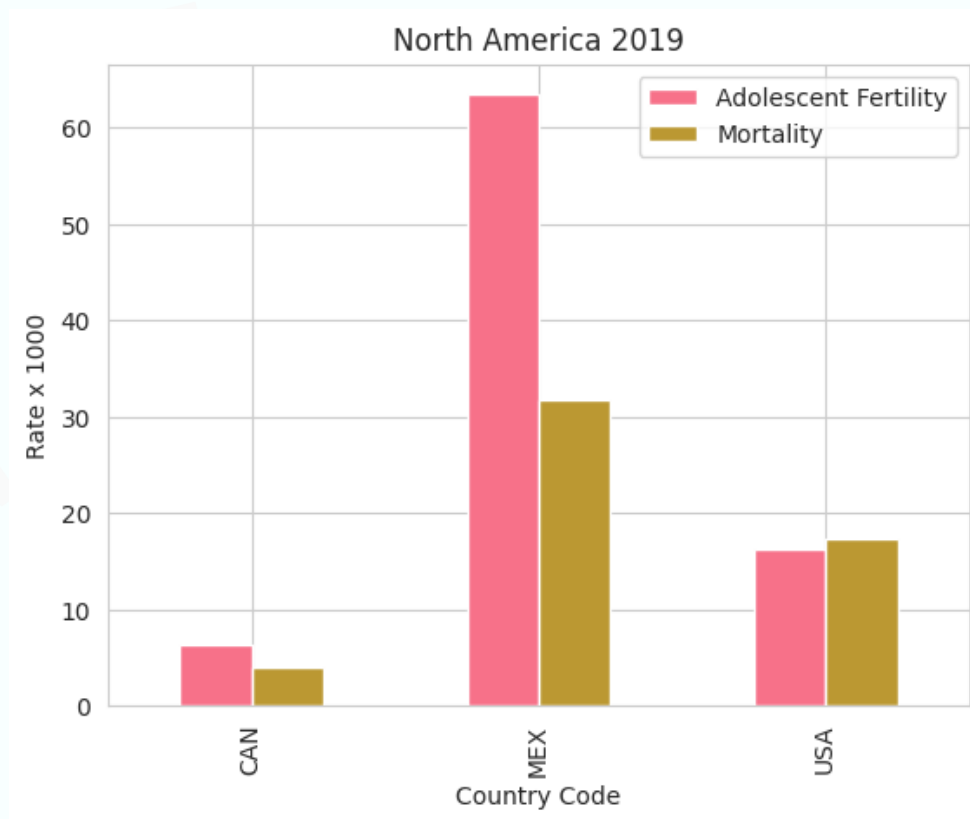


Se ocupó la paleta *husl* y el estilo *whitegrid*.

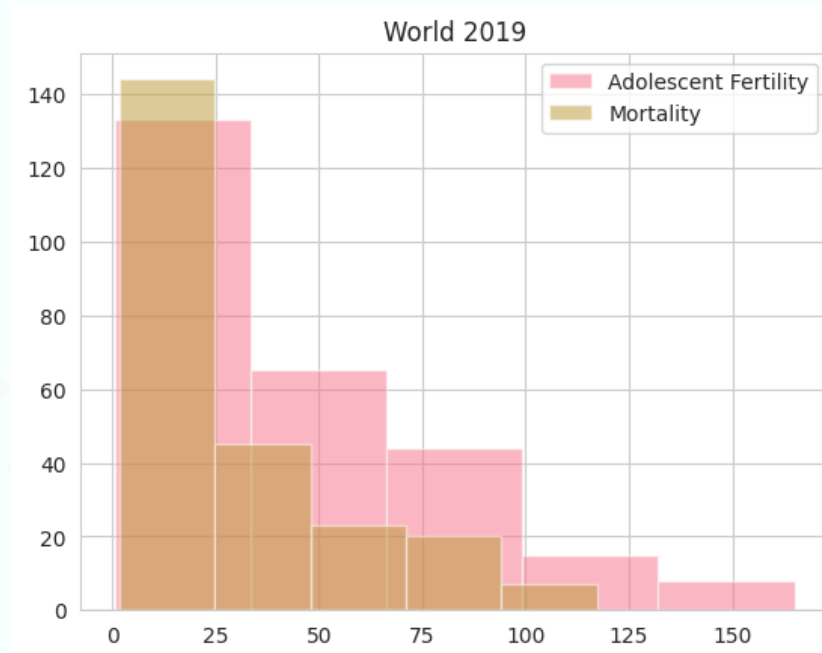
4. Utilizando nuevamente los *dataframes* `adolescent_fertility` y `mortality`, obtén la información de ambos índices para los países de Norteamérica, en el 2019. Haz los cambios necesarios para que quede con la siguiente estructura:

Adolescent Fertility Mortality		
Country Code		
CAN	7.6822	4.9
MEX	58.5434	14.2
USA	17.2524	6.5

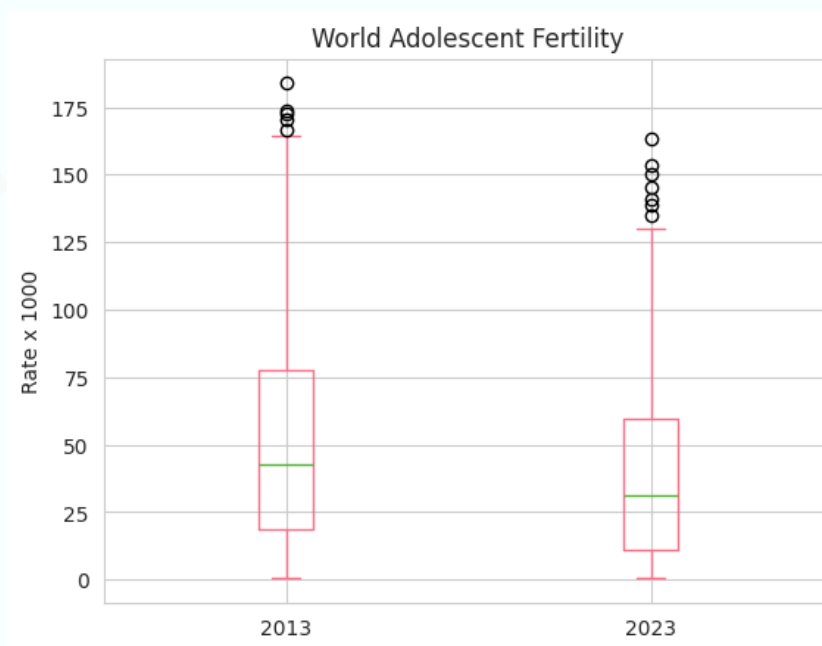
5. Visualiza el *dataframe* anterior en un gráfico de barras, empleando la plataforma pandas. Personaliza sus propiedades para que se vea como en la siguiente imagen:



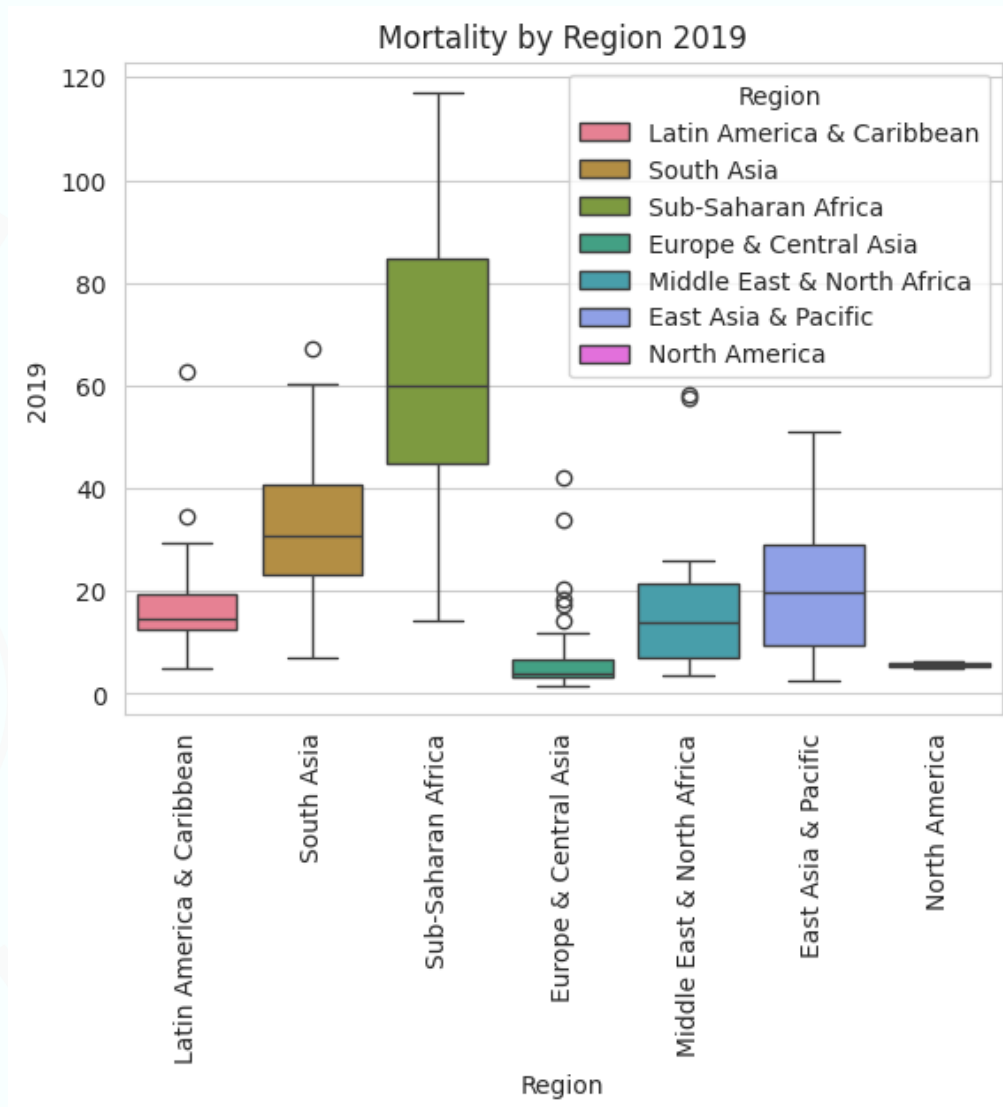
6. Para observar la distribución de ambos indicadores en el mundo, para el 2019, construye un gráfico de histogramas independientes empleando matplotlib. Utiliza 5 *bins* y transparencia de 0.5.



7. Cómo ha cambiado el índice de fecundidad adolescente en la última década? Compáralos con boxplots en los años 2013 y 2023.



8. Combina el *dataframe* `mortality` con el resultante de leer el contenido del archivo **Metadata.csv** (ubicado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente). Utiliza *seaborn* para generar *boxplots* por región a partir del índice del 2019. Asegúrate de lograr el cambio de colores como se muestra en la figura:



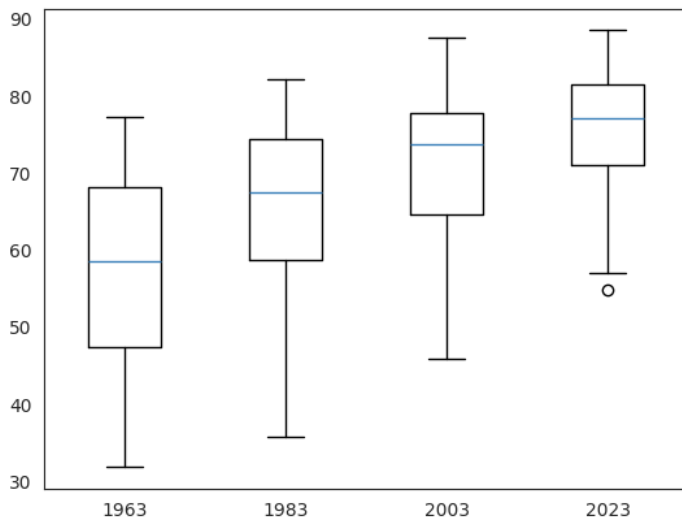
Si tienes dudas para obtener algún resultado, consulta las soluciones en el archivo ubicado en el apartado “Archivos adjuntos” de la plataforma del recurso correspondiente.

Una vez que hayas finalizado el ejercicio, continúa con la segunda sección.

Sección 2

Te invitamos a analizar las preguntas planteadas, así como sus respectivas respuestas.

1. Continuando con el análisis de la expectativa de vida por género que está realizando la ONG, imagina que estás empleando un diagrama de *boxplots* para revisar los cambios en la expectativa de vida de las mujeres cada 20 años y también has calculado los cuartiles de manera numérica. ¿A qué años pertenecen cada uno de los siguientes cuartiles?



- 1er cuartil (Q1) = 71
- 2do cuartil (Q2) = 59
- 3er cuartil (Q3) = 74

- a.** Q1 - 2023, Q2 - 1963, Q3 - 1983
- b.** Q1 - 1983, Q2 - 1983, Q3 - 1963
- c.** Q1 - 2023, Q2 - 1983, Q3 - 2003

Retroalimentación

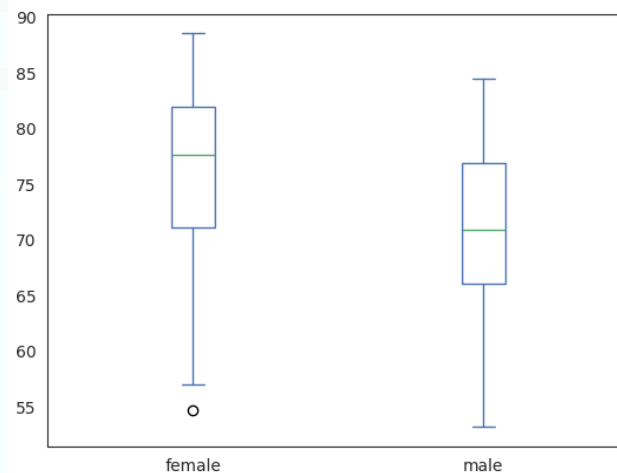
El 2º cuartil es la mediana, que está representada en color azul; el 1º cuartil está por debajo de la mediana y el 3º por encima.

2. Ahora quieres visualizar claramente la diferencia entre hombres y mujeres examinando la dispersión y la tendencia central de ambos conjuntos. Para ello, has decidido generar un diagrama de *boxplots* a partir del siguiente *dataframe* (*lifexp*):

	Country	Code	female	male	Name	Continent	Region
0	ABW		78.779	73.702	Aruba	Americas	Latin America and the Caribbean
1	AFG		67.536	64.467	Afghanistan	Asia	Southern Asia
2	AGO		67.144	62.099	Angola	Africa	Sub-Saharan Africa
3	ALB		81.446	77.728	Albania	Europe	Southern Europe
...	...		...	...	...	...	...
211	YEM		71.388	67.235	Yemen	Asia	Western Asia
212	ZAF		69.599	62.609	South Africa	Africa	Sub-Saharan Africa
213	ZMB		68.673	63.936	Zambia	Africa	Sub-Saharan Africa
214	ZWE		65.013	60.226	Zimbabwe	Africa	Sub-Saharan Africa

215 rows × 6 columns

¿Con qué código lograrías reproducir el siguiente diagrama?

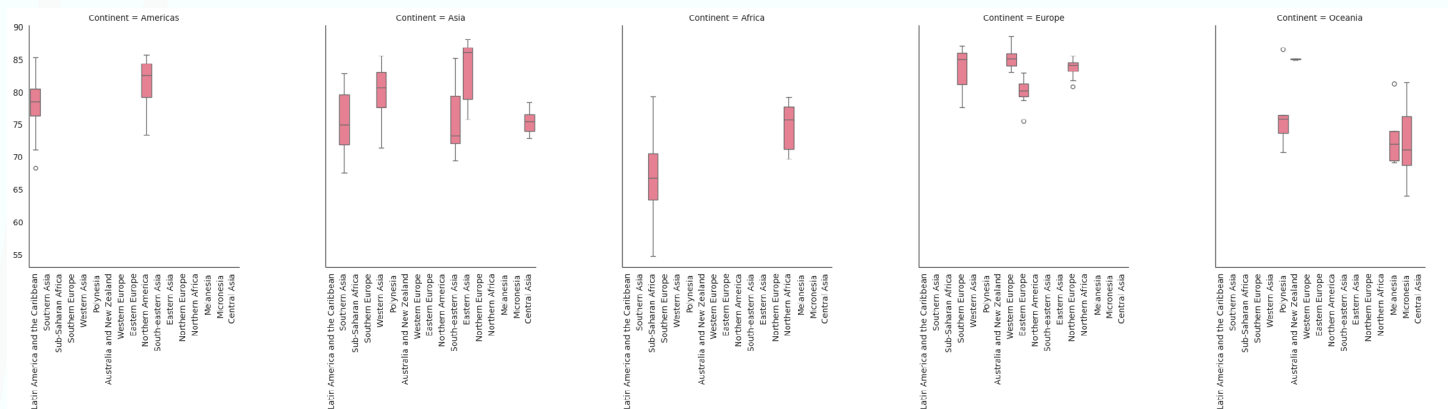


- `lifexp['female'].plot(kind='box')`
`lifexp['male'].plot(kind='box')`
- `plt.boxplot(x=['female'.dropna(), 'male'.dropna()], labels=['female', 'male'])`
- `lifexp[['female', 'male']].plot.box()`

Retroalimentación

Se grafican ambas columnas en la misma función de trazado y con ello los *boxplot* no se superponen.

3. Para generar un gráfico de *boxplots* del indicador de expectativa de vida de las mujeres a partir del *dataframe* `lifexp`, ¿qué código de *seaborn* deberías emplear para separar los resultados por región y continente, como se muestra en la figura?



- `sns.boxplot(x='Region', y='female', col='Continent', data=lifexp)`
- `sns.catplot(kind='box', x='Region', y='Continent', col='female', data=lifexp)`
- `sns.catplot(kind='box', x='Region', y='female', col='Continent', data=lifexp)`

Retroalimentación

La interfaz `catplot` incorpora el parámetro `col` para crear gráficos independientes por continente. La región se especifica en el eje **X**; los valores del indicador, en **Y**.

Ideas para llevar

- Identificaste otros tipos de gráficos que te permitirán complementar los análisis que ejecutabas en lecciones anteriores; y aunque existen muchos tipos más (violinplot, jointplot y swarmplot), estos cubren una amplia gama de visualizaciones que te ayudarán a comprender tus datos.
- Revisaste que las tres plataformas consideradas poseen varias opciones para variables categóricas o cuantitativas y para análisis univariado o bivariado.
- Comprendiste que la visualización es parte fundamental del análisis exploratorio de datos, pero no existe un procedimiento estandarizado a seguir, por lo que deberás estar familiarizado con la información analizada para hacerte las preguntas correctas que te guíen a lo largo del proceso. Todos los conjuntos de datos son diferentes, por lo que la forma de explorarlos también deberá ser diferente.
- Aprendiste que los histogramas y los *boxplots* son representaciones gráficas esenciales para explorar y comprender conjuntos de datos desde diferentes perspectivas. Mientras los histogramas permiten observar la forma general de la distribución y detectar concentraciones o vacíos de datos, los *boxplots* ofrecen una visión estructurada de la dispersión y posibles valores atípicos. Juntos proporcionan una base sólida para el análisis inicial, permitiendo al analista tomar decisiones más informadas y plantear hipótesis con mayor claridad. Su combinación en el análisis exploratorio aporta una visión complementaria que enriquece la interpretación de los datos antes de aplicar métodos estadísticos más avanzados.

Material adicional

- **Matplotlib**

The Matplotlib development team. (2025). *Matplotlib: Visualization with Python* [Matplotlib: Visualización con Python]. Matplotlib. Recuperado el 25 de abril de 2025 de: <https://matplotlib.org>

- **Pandas**

Pandas via. (2025). *Pandas*. Recuperado el 25 de abril de 2025 de: <https://pandas.pydata.org>

- **Seaborn**

Waskom, M. (2021). *Seaborn: statistical data visualization* [Seaborn: visualización de datos estadísticos]. Seaborn. Recuperado el 25 de abril de 2025 de: <https://seaborn.pydata.org>

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.

Glosario

- **Boxplots:** Se utilizan para resumir la distribución de una variable cuantitativa mediante cinco estadísticas clave (mínimo, primer cuartil, mediana, tercer cuartil y máximo), permitiendo identificar la dispersión, la simetría y los valores atípicos en los datos.
- **Histogramas:** Se utilizan para representar la distribución de variables cuantitativas continuas, agrupando los datos en intervalos (*bins*) y mostrando la frecuencia de observación de cada uno mediante barras contiguas.